

**A Three-Stage
Decomposition Approach to
High School Timetabling
Using Mixed Integer
Programming**

Ashe Raymond-Barker

Year 4 Project
School of Mathematics
The University of Edinburgh
March 6, 2025

Abstract

The high school timetabling problem involves assigning events to timeslots and resources subject to a range of constraints. This dissertation focuses on generating high-quality feasible solutions within limited computational time. The inclusion of resource allocation significantly increases the size of mixed integer programming models and, consequently, their computational difficulty.

To address this, a three-stage decomposition approach is proposed. The method separates the problem into (i) a feasibility stage for event–time assignment, (ii) an optimisation stage incorporating soft time-based constraints, and (iii) a room allocation stage. This decomposition reduces the number of decision variables in each model and exploits the structure of the problem, where most constraints primarily affect time assignment rather than resource allocation.

The approach is evaluated on benchmark instances from the XHSTT dataset and compared against monolithic and two-stage formulations. Results show that the proposed method produces smaller models and yields higher-quality feasible solutions within fixed time limits. In particular, the decomposition enables the solution of instances that are otherwise computationally prohibitive under integrated formulations.

These findings demonstrate that multi-stage decomposition is an effective strategy for managing the complexity of event–resource–time timetabling problems.

Please see <https://github.com/Ashe2002/AutomaticTimetabling> for code relevant to this dissertation

Declaration

I

(*Ashe Raymond-Barker*) declare that this thesis is my own work. The following Generative AI tools were used in the completion of this work. ChatGPT was utilised to assist in developing a parser for extracting data instances from XML files, and to support the creation of graphs and figures. It was also consulted, alongside Overleaf suggestions, to improve the academic tone and clarity of the writing.

Acknowledgements

Thank you to my supervisor, Lars, for all of his help with this dissertation!

Contents

Abstract	ii
Acknowledgements	iv
Contents	vi
1 Introduction	1
2 Literature Review	2
2.1 Educational Timetabling: Problem Domains	2
2.2 Research Imbalance and the XHSTT Benchmark	3
2.3 Different Approaches to HSTT	3
2.4 Advances in MIP for UCTT	4
2.5 Research Gap and Motivation	4
I An Introduction to Mixed Integer Programming	6
3 Formulating Mixed Integer Programming Problems	7
3.1 Problem Classes	7
3.2 Feasibility and Optimisation Problems	8
3.3 Formulating MIP Feasibility Problems	9
3.4 From Feasibility to Optimisation	12
3.5 Hard and Soft Constraints	14
4 Solving Mixed Integer Programming Problems	16
4.1 MIP Relaxation	16
4.1.1 LP Relaxation	16
4.1.2 QP Relaxation	19
4.2 Branch and Bound	21
4.2.1 Motivation	21
4.2.2 Method	22
4.3 MIP Solvers	25
4.4 MIP Difficulties	26

II	High School Timetabling	28
5	The High School Timetabling Problem	29
5.1	Problem Definition	29
5.2	Problem Modelling and Objectives	30
5.3	The XHSTT Dataset	31
5.3.1	Notation and Problem Structure	32
5.3.2	Constraints and Objective Function	33
6	Event-Time Scheduling Problems	35
6.1	Problem Description	35
6.2	Baseline Model	36
6.2.1	Basic Feasible Constraint	36
6.2.2	Instance Specific Constraints	36
6.3	Proposing a Two-Stage Model	38
6.4	Events Spanning Multiple Timeslots	39
6.5	Feasible Times	42
6.6	Two-Stage Model Analysis	44
6.7	Gurobi Settings	46
6.8	Updated Model	48
7	Event-Resource-Time Scheduling Problems	49
7.1	Problem Description and MIP Formulation	49
7.2	Occupancy Formulations	50
7.3	3-Stage Decomposition	53
7.3.1	The Importance of Room Allocation Constraints	56
7.4	Final Model	56

Chapter 1

Introduction

Timetabling is a fundamental optimisation problem with applications across a wide range of domains, including education [10], transportation [6], and sports [25] scheduling. The need to allocate limited resources efficiently under a set of constraints makes it a problem of significant practical importance.

The field developed alongside operational research, linear programming, and optimisation theory during the Second World War, when large-scale, time-critical, and resource-constrained decision-making became essential. Early applications focused on improving the effectiveness of military operations, leading to measurable gains and establishing operational research as a recognised discipline. In the post-war period, these techniques were increasingly applied to industrial problems such as workforce scheduling and delivery planning. [24]

With advances in computing, timetabling has evolved into a combinatorial optimisation problem, typically incorporating both hard constraints, which must be satisfied, and soft constraints, which aim to improve the quality of a solution. Today, it remains an active area of research, particularly in domains such as university course scheduling and exam timetabling, where increasingly complex requirements continue to drive the need for efficient and robust solution methods. [3]

Chapter 2

Literature Review

2.1 Educational Timetabling: Problem Domains

Educational timetabling is typically divided into three main domains: High School Timetabling (HSTT), University Course Timetabling (UCTT), and University Examination Timetabling (ETT) [26]. While all aim to allocate events to times and resources under constraints, each domain has distinct structural characteristics that influence solution approaches.

High School Timetabling concerns the weekly scheduling of lessons within a school. Students are usually organised into fixed classes, and teachers and classes are often predefined for each lesson. The task is therefore to assign lessons to timeslots and rooms while avoiding resource conflicts. [22]. Additional constraints often include limiting gaps in student schedules and distributing subjects evenly across the week. [13]

University Course Timetabling differs in both structure and scale. Students enrol individually in modules, leading to more complex and less rigid conflict relationships. Timetables may span multiple weeks, often requiring recurring events to be scheduled consistently. The primary challenge lies in the large number of students, courses, and rooms, with the objective typically focused on minimising clashes between events with shared students. [26]

University Examination Timetabling involves scheduling one-off exams within a limited time window. The key requirement is to avoid student conflicts, while also optimising fairness by, for example, reducing consecutive exams or multiple exams in a single day. [26]

These differences lead to varying problem structures. HSTT and ETT often exhibit dense conflict relationships due to limited resources and tight scheduling windows, whereas UCTT tends to produce larger but sparser conflict graphs driven by student enrolment patterns. Although these problems belong to the same class of combinatorial optimisation problems, their structural differences significantly influence the choice and effectiveness of solution techniques.

2.2 Research Imbalance and the XHSTT Benchmark

University Course Timetabling (UCTT) is one of the most extensively studied combinatorial optimisation problems in the timetabling literature [15]. Over the past two decades, it has been the focus of multiple International Timetabling Competitions (ITC 2002, 2007, and 2019), which introduced standardised benchmark datasets and stimulated significant advances in solution methods [16]. As a result, a large body of work exists spanning constraint programming, heuristic, and exact approaches [3].

University Examination Timetabling (ETT) has similarly received substantial attention. Research in this area has been driven by widely adopted benchmark datasets, most notably the Carter examination instances [2], and later the ITC 2007 examination track. These benchmarks have been reused extensively, with surveys reporting over one hundred studies based on the Carter instances alone [23].

In contrast, High School Timetabling (HSTT) has received comparatively less attention [22], largely due to the historical lack of standardised benchmark datasets. This gap was addressed by the introduction of the XHSTT benchmark for the International Timetabling Competition 2011. The XHSTT archive contains over 40 real-world instances from diverse educational systems, providing a unified format for modelling and evaluation.

This dissertation uses a selection of XHSTT instances to address the ITC 2011 challenge of assigning lessons to timeslots and rooms while avoiding resource conflicts. A key strength of the benchmark is its diversity: instances vary significantly in structure, including differences in student organisation, lesson durations, and constraint priorities.

However, this diversity also introduces challenges, as methods that perform well on one instance may not generalise to others. Therefore, effective solutions will demonstrate flexibility and robustness across varying problem structures.

2.3 Different Approaches to HSTT

The ITC 2011 competition was dominated by heuristic approaches [22]. The winning team, Team GOAL (Brazil), employed a hybrid metaheuristic, combining Iterated Local Search, Large Neighbourhood Search, and adaptive move selection. Other successful approaches included Tabu Search, Simulated Annealing, and genetic algorithm hybrids. This dominance can be largely attributed to the competition’s 1000-second time limit, which favoured methods capable of quickly producing high-quality feasible solutions over exact optimisation techniques, which risked failing to find any feasible solution.

Following the competition, research began to explore exact methods for HSTT. Kristiansen, Sørensen, and Stidsen proposed the first general integer programming formulation capable of solving arbitrary XHSTT instances, demonstrating competitive performance even under strict time limits [14]. Their approach used a two-stage model: first solving a feasibility problem using only hard constraints,

then optimising with both hard and soft constraints using the initial solution as a warm start.

Subsequent work by Fonseca et al. [7] improved this formulation by strengthening the linear relaxation through additional valid inequalities, reducing optimality gaps and enhancing solution quality, albeit with a modest increase in computational effort.

2.4 Advances in MIP for UCTT

Much of the progress in applying Mixed Integer Programming (MIP) to educational timetabling has been driven by the University Course Timetabling Problem (UCTT). The ITC 2019 competition [16] introduced large-scale benchmark instances significantly more complex than those in XHSTT, requiring carefully designed formulations to ensure computational feasibility.

Notably, the most successful approaches employed MIP as a core component. The winning method used a graph-based formulation that exploited conflict structures between events, incorporating clique and cover inequalities to strengthen the model [10]. A preprocessing phase further reduced the problem size by eliminating infeasible assignments.

To improve scalability, the approach adopted a decomposition strategy: events were first assigned to timeslots, and rooms were allocated in a second stage. This separation significantly reduced model complexity. Solution quality was further enhanced through an iterative refinement process, where subsets of problematic events were re-optimised using restricted MIP models.

These results demonstrate that, when combined with effective preprocessing and decomposition, MIP-based methods can scale to large real-world timetabling problems. Such advances in UCTT provide valuable insights for the development of more robust approaches to HSTT.

2.5 Research Gap and Motivation

High School Timetabling (HSTT) represents a combined feasibility and optimisation problem that, relative to other domains in educational timetabling, has received comparatively less attention in the development of exact optimisation techniques [21]. While the introduction of the XHSTT benchmark has provided a diverse collection of practically relevant, real-world instances, the majority of research it has stimulated has focused on heuristic and metaheuristic approaches. Although these methods are effective at producing high-quality solutions, particularly under strict time constraints, they do not provide quantitative guarantees regarding solution optimality or proximity to optimal solutions.

This imbalance highlights a clear research opportunity. Significant advances in Mixed Integer Programming (MIP) formulations for the University Course Timetabling Problem have demonstrated that carefully designed exact models, supported by preprocessing and decomposition techniques, can scale to large and complex instances. However, the extent to which these methods can be adapted

to the structurally different and diverse HSTT domain remains overlooked.

Accordingly, this dissertation investigates the application of a MIP-based approach to instances of the HSTT problem. In particular, it focuses on the development and evaluation of formulation strength, computational performance, and solution quality across a selection of XHSTT benchmark instances subject to a time limit of 10 minutes. By doing so, it aims to evaluate the viability of MIP techniques within this domain and to contribute to a deeper understanding of effective problem formulation strategies.

Part I

An Introduction to Mixed Integer Programming

Chapter 3

Formulating Mixed Integer Programming Problems

The following two chapters introduce the fundamental concepts of Mixed Integer Programming (MIP). They provide the background required to understand the formulation and solution of the High School Timetabling (HSTT) problem presented in later chapters.

3.1 Problem Classes

Definition 3.1. [11] *A **Mixed Integer Programming (MIP)** problem is a feasibility or optimisation problem characterised by the following components:*

- **Decision Variables:** Continuous, integer or binary variables representing choices to be made
- **Constraints:** A set of conditions or rules that restrict the feasible values taken by decision variables
- **Objective Function:** A function that must be minimised or maximised

Within the scope of MIP problems, we focus on two specific subclasses that are particularly relevant to High School Timetabling (HSTT) problems: Mixed Integer Linear Programming (MILP) and Mixed Integer Quadratic Programming (MIQP).

Definition 3.2. [4] *A **Mixed Integer Linear Programming (MILP)** problem is a MIP problem of the form:*

$$\begin{aligned} \min / \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I \\ & x_i \in \mathbb{R} \quad \forall i \notin I \end{aligned} \tag{3.1.1}$$

where $c^\top = (c_1, \dots, c_n)$, $x^\top = (x_1, \dots, x_n)$, and $b^\top = (b_1, \dots, b_m)$. The matrix $A = (a_{ij})$ is of dimension $m \times n$. We let $I \subseteq \{1, \dots, n\}$ denote the index set of

decision variables that are restricted to take integer values. All entries of c , A , and b are assumed to be rational numbers and we assume that $n \geq 1$.

In this formulation, $x = (x_1, \dots, x_n)$ represents the decision variables. The objective function is given by $f(x) = c^\top x$ subject to the linear constraints $Ax \leq b$, $x_i \in \mathbb{Z} \quad i \in I$ and $x_i \in \mathbb{R} \quad i \notin I$.

Remark. The assumption $n \geq 1$ ensures that the problem contains at least one integer decision variable. If no such variables were present, the problem would reduce to a Linear Programming (LP) problem.

Definition 3.3. [5] A **Mixed Integer Quadratic Programming (MIQP)** problem is a MIP problem of the form:

$$\begin{aligned} \min / \max \quad & x^\top Qx + c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I \\ & x_i \in \mathbb{R} \quad \forall i \notin I \end{aligned} \tag{3.1.2}$$

where $Q \in \mathbb{R}^{n \times n}$ defines the quadratic coefficients in the objective function and the remaining notation is identical to the MILP formulation described previously in Definition 3.2.

Remark. In this formulation, the objective function is $f(x) = x^\top Qx + c^\top x$ which contains a quadratic term $x^\top Qx$ in addition to the linear component. The feasible region is defined by the same linear constraints and integrality requirements as in the MILP formulation.

MILP and MIQP problems share the same linear constraint structure and may include both continuous and integer decision variables. They differ in the form of their objective functions: MILP problems have a linear objective function, whereas MIQP problems include a quadratic term in addition to the linear component.

3.2 Feasibility and Optimisation Problems

Mixed Integer Programming problems can be classified as either feasibility problems or optimisation problems. To formalise this distinction, we first define feasible solutions and feasible regions.

Definition 3.4. [18] A **feasible solution** to a MIP problem is a vector of decision variable values that satisfy all constraints, including any integrality requirements.

Definition 3.5. [18] The **feasible region** of a Mixed Integer Programming problem is the set of all feasible solutions

Definition 3.6. [4] A **Feasibility Problem** seeks to find any solution that satisfies the constraints of the problem, without the need to optimise an objective function.

Definition 3.7. [19] An **Optimisation Problem** seeks to optimise an objective function subject to constraints. Formally, this can be written as

$$\min_{x \in X} f(x) \quad \text{or} \quad \max_{x \in X} f(x)$$

where $X \subseteq \mathbb{R}^n$ is the feasible region and $f : X \rightarrow \mathbb{R}$ is the objective function.

Definition 3.8. [19] An **optimal solution** to an optimisation problem is a feasible solution $x^* \in X$ such that

$$f(x^*) \leq f(x) \quad \forall x \in X$$

for a minimisation problem and

$$f(x^*) \geq f(x) \quad \forall x \in X$$

for a maximisation problem

Referring to Definitions 3.2 and 3.3, a MIP feasibility problem can be viewed as a special case of problem in which the objective function is constant (for example $f(x) = 0$). Hence, any all feasible solutions are also optimal. MIP optimisation problems extend feasibility problems by introducing a function that assigns a value to each feasible solutions. The goal is therefore not just to find a feasible solution, but to identify one which optimises this function over the feasible region.

3.3 Formulating MIP Feasibility Problems

A fundamental challenge of Mixed Integer Programming is translating real-world decision problems into mathematical models that can then be solved using optimisation techniques. The following examples illustrate how practical problems can be formulated as MIP feasibility problems. These examples serve as a foundation for modelling the High School Timetabling (HSTT) problem presented in later chapters.

Example 3.9 (Product Mix Problem 1). *An Edinburgh based carpenter builds tables and chairs for a school. Each Table uses 2 crates of wood and takes 1 day to construct. Chairs are smaller but more difficult to build, and therefore only require 1 crate of wood, but take 2 days to construct. The carpenter has 24 crates of wood at his disposal and must build at least one chair for every table he produces. He wants to know a feasible number of chairs and tables that he can build in the next 30 days. To do this, he asks you to formulate the feasibility problem.*

Solution. Let us begin by visualising this information in a table:

	Wood crates used	Construction time (days)
Table	2	1
Chair	1	2

Table 3.1: Table showing resource and time costs of building a product

Let $x_t \in \mathbb{Z}_{\geq 0}$ be the number of tables we build and $x_c \in \mathbb{Z}_{\geq 0}$ be the number of chairs we build. These are our decision variables. Next we define our constraints.

- The carpenter has at most 24 crates of wood that he can use:

$$2x_t + x_c \leq 24$$

- All tables and chairs must be built within the next 30 days:

$$x_t + 2x_c \leq 30$$

- At least one chair must be built for every table that is produced:

$$x_c \geq x_t$$

Finally, we define our objective. Since we are only interested in feasibility, the goal is simply to find any value of (x_t, x_c) that satisfies all constraints. So we can set $f(x_t, x_c) = 0$.

Therefore, the feasibility problem can be written as:

$$\begin{aligned} \max \quad & f(x_t, x_c) = 0 \\ \text{s.t.} \quad & 2x_t + x_c \leq 24 \\ & x_t + 2x_c \leq 30 \\ & x_c \geq x_t \\ & x_t, x_c \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

Note. Although constraints may be expressed using $\leq$, $\geq$, or $=$, many optimisation models are written in a standard inequality form $Ax \leq b$, together with non-negativity constraints on the variables. Constraints in other forms can be equivalently transformed into this representation. For example:

$$x_t \geq 5 \quad \Leftrightarrow \quad -x_t \leq -5,$$

and

$$x_t = 10 \quad \Leftrightarrow \quad x_t \leq 10, x_t \geq 10 \quad \Leftrightarrow \quad x_t \leq 10, -x_t \leq -10.$$

Many optimisation solvers perform such transformations automatically during preprocessing. For clarity, we will continue to write constraints using the forms $\leq$, $\geq$, or $=$.

Example 3.10 (Allocation Problem 1). A school has 2 classes, A and B, with sizes of 18 and 21 respectively. They also have two rooms, α and β , with capacities

of 30 and 25 respectively. Each room can hold at most one class. The school wishes to know whether there is a feasible allocation of rooms to classes that satisfies all requirements. They ask you to formulate the feasibility problem to help them do this.

Solution. It is obvious that there is a feasible allocation of classes to rooms for this problem. However, formulating a MIP problem from the information is a more difficult task. Presenting the information in tables, we have:

Class	Class Size
A	18
B	21

Table 3.2: Class sizes

Room	Room Capacity
α	30
β	25

Table 3.3: Room capacities

We need to decide which class we assign to which room. Let us then define a binary decision variable, $x_{ij} \in \{0, 1\}$, where:

$$x_{ij} = \begin{cases} 1 & \text{if class } i \text{ is assigned to room } j \\ 0 & \text{otherwise} \end{cases}$$

We express the constraints as follows:

- Each class must be allocated to a room:

$$x_{A\alpha} + x_{A\beta} = 1, \quad x_{B\alpha} + x_{B\beta} = 1$$

- Each room can be assigned at most 1 class:

$$x_{A\alpha} + x_{B\alpha} \leq 1, \quad x_{A\beta} + x_{B\beta} \leq 1$$

Given this is a feasibility problem, we set the objective function to be $f(x_{ij}) = 0$ so that any feasible solution is optimal. Compactly, the feasibility problem can be written as:

$$\begin{aligned} \max \quad & f(x_{ij}) \\ \text{s.t.} \quad & x_{A\alpha} + x_{A\beta} = 1 \\ & x_{B\alpha} + x_{B\beta} = 1 \\ & x_{A\alpha} + x_{B\alpha} \leq 1 \\ & x_{A\beta} + x_{B\beta} \leq 1 \\ & x_{ij} \in \{0, 1\}. \end{aligned}$$

Despite the differences between Examples 3.9 and 3.10, they both follow a common modelling procedure:

1. Identify the decision variables and specify integrality or binary restrictions
2. Define constraints to enforce resource limitations and relationships between variables

3. Identify the objective function

3.4 From Feasibility to Optimisation

The following examples illustrate how feasibility models can be extended to optimisation models by introducing an objective function that assigns a value to each feasible solutions.

Example 3.11 (Product Mix Problem 2). *The carpenter from Example 3.9 now wants to sell the tables and chairs he builds to the school. He knows that he can sell each table for £115 and each chair for £55. He also knows that the school does not have space for any more than 8 tables and 14 chairs. The carpenter now wants to know how many tables and chairs he should build in order to maximise profit. He asks you to formulate the optimisation problem to help him decide.*

Solution.

Our table now reads:

	Price (£)	Wood crates used	Construction time (days)
Table	115	2	1
Chair	55	1	2

Table 3.4: Table showing price, and resource and time costs of building a product

The carpenter is still deciding how many tables and chairs to build, so our decision variables remain $x_t \in \mathbb{Z}_{\geq 0}$ and $x_c \in \mathbb{Z}_{\geq 0}$.

However, We now seek not only a feasible solution, but the optimal one. The goal is to make as much profit as possible and so our objective becomes

$$\max \quad 115x_t + 55x_c$$

Furthermore, given that we know the carpenter will not sell more than 14 chairs or 8 tables (and therefore will not make any profit from doing so), we can impose these as additional constraints, thereby reducing the feasible region of the model: $x_t \leq 8$ and $x_c \leq 14$.

Thus our optimisation problem is:

$$\begin{aligned} \max \quad & 115x_t + 55x_c \\ \text{s.t.} \quad & 2x_t + x_c \leq 24 \\ & x_t + 2x_c \leq 30 \\ & x_t \leq x_c \\ & x_t \leq 8 \\ & x_c \leq 14 \\ & x_t, x_c \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

Example 3.12 (Allocation Problem 2). *The school from Example 3.10 not only wants to allocate classes in a feasible manner, but also in a way that minimises the maximum unused room capacity*

Solution. In this example, the extension from a feasibility problem to an optimisation problem is less immediate. The key step is to compute the difference between each room capacity and class size using the available data. We now create an additional table denoting the unused capacity (room capacity - class size) of different combinations of rooms and classes:

Class	Class Size
A	18
B	21

Table 3.5: Class sizes

Room	Room Capacity
α	30
β	25

Table 3.6: Room capacities

	α	β
A	12	7
B	9	4

Table 3.7: Unused capacity

The objective is to minimise the maximum unused room capacity across all assignments. We can model this with the introduction of an auxiliary variable, $z \geq 0$, that captures the maximum unused capacity. Then the objective function becomes:

$$\min z$$

To ensure that z represents the maximum capacity shortfall, we require that z be at least as large as the as the unused capacity for each class under any assignment.

$$z \geq 12x_{A\alpha} + 7x_{A\beta}$$

$$z \geq 9x_{B\alpha} + 4x_{B\beta}$$

This formulation ensures that z is greater than or equal to all unused capacities, and hence minimising z minimises the maximum unused capacity.

Thus, the optimisation problem can be written as:

$$\begin{aligned} \min \quad & z \\ \text{s.t.} \quad & x_{A\alpha} + x_{A\beta} = 1 \\ & x_{B\alpha} + x_{B\beta} = 1 \\ & x_{A\alpha} + x_{B\alpha} \leq 1 \\ & x_{A\beta} + x_{B\beta} \leq 1 \\ & z \geq 12x_{A\alpha} + 7x_{A\beta} \\ & z \geq 9x_{B\alpha} + 4x_{B\beta} \\ & x_{A\alpha}, x_{A\beta}, x_{B\alpha}, x_{B\beta} \in \{0, 1\}. \end{aligned}$$

Remark. *The introduction of the auxiliary variable z allows the Min–Max objective to be expressed using linear constraints. Without it, the objective would take*

the non-linear form

$$\min \max (12x_{A\alpha} + 7x_{A\beta}, 9x_{B\alpha} + 4x_{B\beta}),$$

3.5 Hard and Soft Constraints

Adding too many constraints, or introducing conflicting constraints, may cause a formulation to become infeasible.

Example 3.13 (Product Mix Problem 3). *Consider Example 3.9. Now imagine that the school requires at least 8 tables and 12 chairs from the carpenter.*

These constraints can be written as $x_t \geq 8$ and $x_c \geq 12$. However, they cannot be satisfied without violating the constraints on available wood and time.

$$2(8) + 1(12) = 28 > 24 \quad (\text{Wood constraint})$$

$$1(8) + 2(12) = 32 > 30 \quad (\text{Time constraint})$$

Hence, there is no feasible solution and no combination of tables and chairs satisfies all constraints. In many real-world optimisation problems, it is therefore useful to distinguish between *hard* constraints and *soft* constraints.

Definition 3.14. [17] A **hard constraint** is a constraint that must be satisfied for a solution to be considered feasible. Any solution violating a hard constraint is rejected.

Definition 3.15. [17] A **soft constraint** is a constraint that may be violated, but where each violation incurs a penalty in the objective function. The goal is typically to minimise the total penalty associated with such violations.

Example 3.16 (Product Mix Problem 4). *Consider Example 3.13. Suppose the carpenter and school reach an agreement by which the carpenter will incur a penalty of £30 for every table short of 8 he produces and £10 for every chair short of 12. The carpenter now wants to know how this affects the optimisation formulation.*

Solution.

Instead of enforcing these requirements as hard constraints, we incorporate them into the objective function as soft constraints. We introduce slack variables $s_t, s_c \geq 0$ to measure how far the carpenter falls short of the required production targets. These variables represent the shortfall in tables and chairs respectively.

We then define the soft constraints:

$$s_t \geq 8 - x_t$$

$$s_c \geq 12 - x_c$$

These constraints ensure that each slack variable represents the positive part of the shortfall, i.e. $s_t = \max(0, 8 - x_t)$ and $s_c = \max(0, 12 - x_c)$. Hence, if

the production target is met, the corresponding slack variable can take the value zero. Otherwise, it captures the magnitude of the shortfall.

We then define the new objective function to reflect the penalties attached to the shortfall of each product:

$$\max \quad 115x_t + 55x_c - 30s_t - 10s_c$$

Since the objective function penalises large values of s_t and s_c , the model seeks to minimise shortfalls while balancing production profit. The full optimisation problem with soft constraints is therefore given by:

$$\begin{aligned} \max \quad & 115x_t + 55x_c - 30s_t - 10s_c \\ \text{s.t.} \quad & 2x_t + x_c \leq 24 \\ & x_t + 2x_c \leq 30 \\ & x_c \geq x_t \\ & s_t \geq 8 - x_t \\ & s_c \geq 12 - x_c \\ & x_t, x_c \in \mathbb{Z}_{\geq 0} \\ & s_t, s_c \geq 0 \end{aligned}$$

Example 3.17 (Product Mix Problem 5). *Consider Example 3.16. Assume the carpenter still incurs a penalty fee of £30 for every table short of 8 but now the penalty for not meeting the chair requirement is given by the square of the shortfall in chairs. The carpenter again wants to know how this affects the optimisation formulation.*

Solution. The introduction of a quadratic penalty places a greater emphasis on the magnitude of the shortfall of chairs, as the penalty increases non-linearly with the size of the violation.

We retain the same constraints and slack variables as in Example 3.16. However, the objective function is now given by:

$$\max \quad 115x_t + 55x_c - 30s_t - s_c^2$$

Since the objective function now contains a quadratic term, the problem becomes a Mixed Integer Quadratic Programming problem.

Soft constraints are particularly important in timetabling problems, where satisfying all requirements simultaneously is often impossible. Instead, solutions are evaluated based on the penalties associated with violating desirable but non-essential constraints.

Chapter 4

Solving Mixed Integer Programming Problems

Having formulated a Mixed Integer Programming (MIP) problem, the next step is to determine an optimal solution. Although a variety of solution methods exist, many modern solvers are based on the Branch and Bound framework [11].

In this chapter, we introduce and develop the Branch and Bound framework, before discussing its limitations and additional techniques employed by modern optimisation solvers. To do this, we first introduce the concept of MIP relaxations.

4.1 MIP Relaxation

4.1.1 LP Relaxation

Definition 4.1. [4]

*The Linear Programming relaxation (**LP relaxation**) of a MIP is obtained by removing the integrality constraints on the integer variables, allowing them to take continuous values.*

If a MILP is given by

$$\begin{aligned} \min \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I, \\ & x_i \in \mathbb{R} \quad \forall i \notin I, \end{aligned}$$

then its LP relaxation is obtained by replacing the integrality constraints with

$$x \in \mathbb{R}^n.$$

This concept can be visualised graphically. For example, Figure 4.1 compares the feasible regions of the MILP from Example 3.11 and its LP relaxation. While the MILP restricts solutions to integer points, its LP relaxation allows all real-valued points within the feasible region.

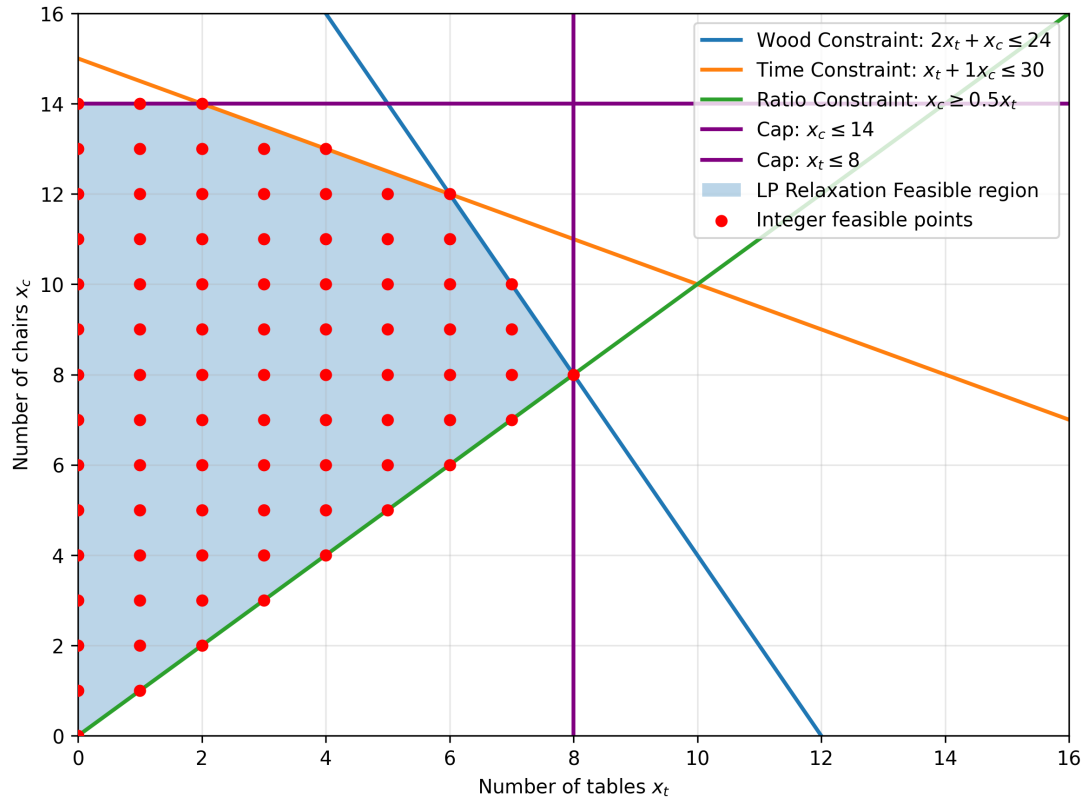


Figure 4.1: Comparison of the feasible regions of the MILP and its LP relaxation. The LP relaxation allows all real-valued points, while the MILP restricts solutions to integer points.

Simple LP problems like this can be solved graphically, and their solutions play an important role in solving the corresponding MILP.

Example 4.2 (Product Mix Problem 2). *The carpenter now wishes to determine the optimal solution to the LP relaxation of the MILP problem we formulated in Example 3.11.*

Solution.

To solve the LP relaxation graphically, we first plot the feasible region determined by the constraints. Next, we consider lines of the form

$$115x_t + 55x_c = k,$$

which represents lines of constant objective value.

Given that this is a maximisation problem, we translate the line parallel to itself in the direction of increasing k . Eventually, this line reaches a final point of intersection with the feasible region. The optimal solution is then given by this point of intersection.

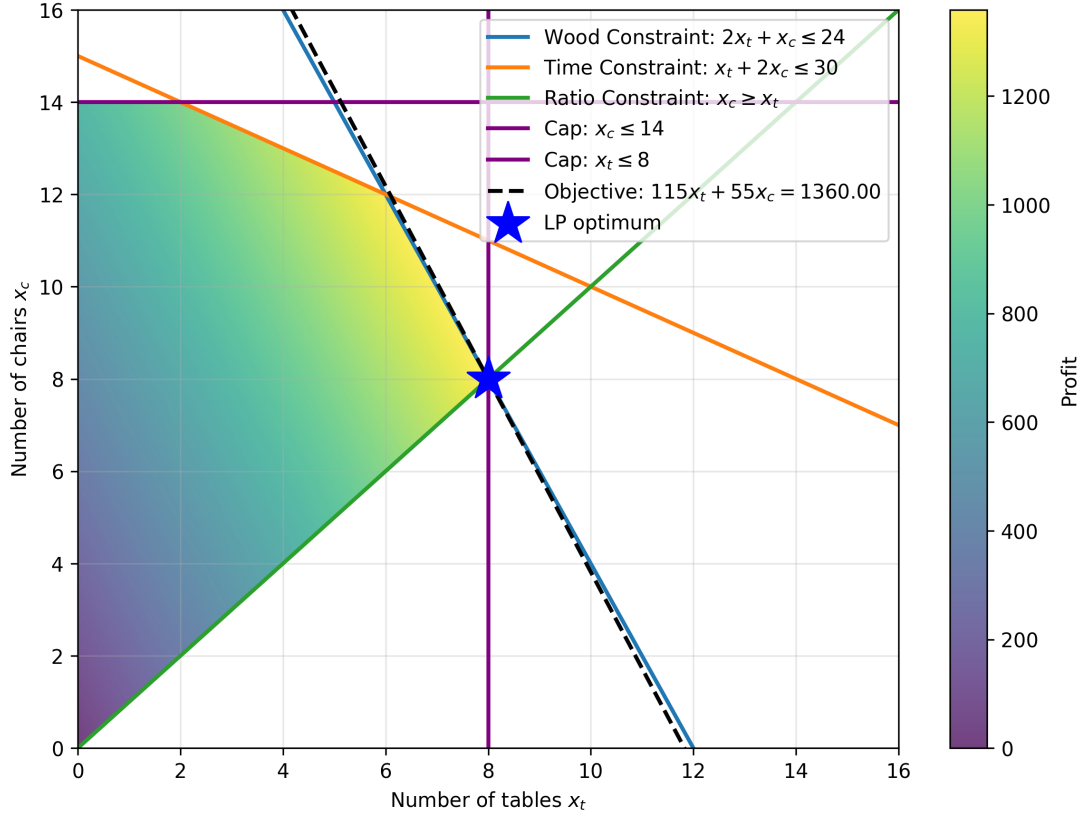


Figure 4.2: Heatmap corresponding to the objective value of feasible region for the LP relaxation of the MIP in Example 3.11

Figure 4.2 shows that the optimum solution to the LP Relaxation is at $(x_t, x_c) = (8, 8)$ with an objective value of

$$115(8) + 55(8) = \text{£}1360$$

The optimal solution therefore occurs at a vertex of the feasible region. This is a property that holds for all linear programming problems, though that vertex may not be a unique optimal solution. [4]

Theorem 4.3. *The optimal objective value of the LP relaxation of a MILP provides a bound on the optimal objective value of the original MILP. In particular, for a maximisation problem, the LP relaxation provides an upper bound, while for a minimisation problem, it provides a lower bound.*

Proof. The feasible region of a MILP is a subset of the feasible region of its LP relaxation, since the LP relaxation removes the integrality constraints. Consequently, in a maximisation (minimisation) problem, the LP relaxation can attain an objective value at least as large (small) as that of the MILP, and therefore provides an upper (lower) bound on the optimal MILP objective value. $\square$

Corollary 4.4. *If the optimal solution to the LP relaxation is integer-valued, then it is also an optimal solution to the original MILP.*

Therefore, in Example 4.2, $(x_t, x_c) = (8, 8)$ is also the solution to the MILP problem. So, to maximise profit, the carpenter should build 8 tables and 8 chairs.

4.1.2 QP Relaxation

Quadratic programming (QP) relaxation is defined in an analogous way to LP relaxation.

Definition 4.5. *The **quadratic programming relaxation (QP relaxation)** of a mixed-integer quadratic programme (MIQP) is obtained by removing the integrality constraints on the integer variables, allowing them to take continuous values.*

If a MIQP is given by

$$\begin{aligned} \min \quad & x^\top Qx + c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I, \\ & x_i \in \mathbb{R} \quad \forall i \notin I, \end{aligned}$$

then its QP relaxation is obtained by replacing the integrality constraints with

$$x \in \mathbb{R}^n.$$

Theorem 4.6. *The optimal objective value of the QP relaxation of a MIQP provides a bound on the optimal objective value of the original MIQP. In particular, for a maximisation problem, the QP relaxation provides an upper bound, while for a minimisation problem, it provides a lower bound.*

This holds by the same reasoning as the proof for Theorem 4.3.

Corollary 4.7. *If the optimal solution to the QP relaxation is integer-valued, then it is also an optimal solution to the original MIQP.*

The difficulty of solving the QP relaxation depends on the properties of the matrix Q . For a minimisation problem, if Q is positive semidefinite ($x^\top Qx \geq 0$ for all $x \in \mathbb{R}^n$), then the quadratic objective function is convex, and the resulting quadratic programme is a convex optimisation problem. For a maximisation problem, the corresponding condition is that Q be negative semidefinite ($x^\top Qx \leq 0$ for all $x \in \mathbb{R}^n$), so that the objective function is concave.

Such problems can be solved efficiently using standard optimisation algorithms (see [1]). If these definiteness conditions do not hold, the resulting problem is non-convex and may be significantly harder to solve.

Figure 4.3 presents a relaxation of the convex MIQP

$$\begin{aligned} \min \quad & x_1^2 + x_2^2 \\ \text{s.t.} \quad & -3 \leq x_1 \leq 3, \\ & -3 \leq x_2 \leq 3, \\ & x_1, x_2 \in \mathbb{Z}. \end{aligned}$$

whilst Figure 4.4 presents the relaxation of the non-convex MIQP

$$\begin{aligned} \min \quad & x_1^2 - x_2^2 + 0.5x_1 \\ \text{s.t.} \quad & -2 \leq x_1 \leq 2, \\ & -2 \leq x_2 \leq 2, \\ & x_1, x_2 \in \mathbb{Z}. \end{aligned}$$

The red dots in the contour graphs represent local minima. In the convex case, there is a unique global minimum, whereas in the non-convex case multiple local minima are present. Unlike convex problems, non-convex quadratic programmes may converge to different locally optimal solutions depending on the starting point, and there is no guarantee of obtaining a global optimum. This significantly increases the computational difficulty of solving the relaxation, as global optimisation techniques must be employed.

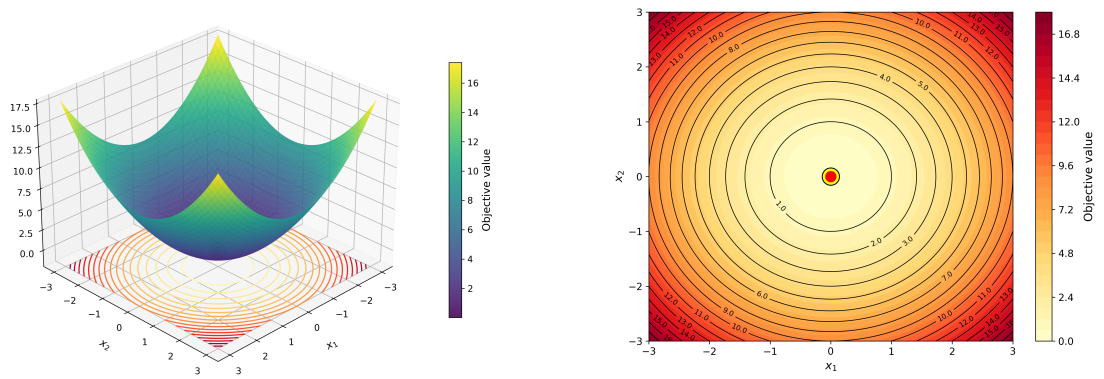


Figure 4.3: Relaxation of a Convex MIQP

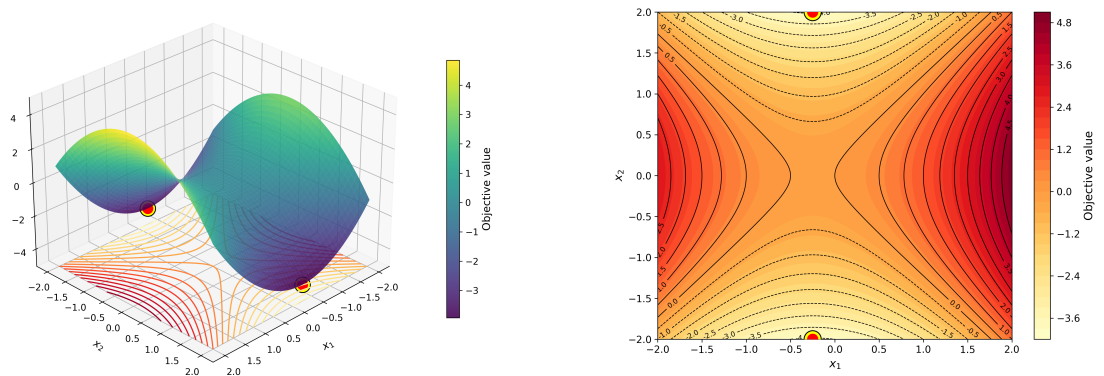


Figure 4.4: Relaxation of a Non-convex MIQP

Note. In the timetabling problems we consider in later chapters, soft constraint violations are penalised using weighted linear or quadratic functions of the form $w_k d_k$ or $w_k d_k^2$, where $w_k \geq 0$ and d_k are deviation variables. Each quadratic term is convex, and since convexity is preserved under summation, the overall objective

function is also convex. Combined with linear constraints, this yields a convex quadratic programming (QP) relaxation.

4.2 Branch and Bound

4.2.1 Motivation

In the Product Mix Problem from Example 4.2, the optimum solution of the LP relaxation happened to also be the optimum solution of the MILP problem. However, the optimum solution of the LP/QP relaxation will often contain non-integer variables. In such cases, we cannot simply round the fractional values to the nearest integers. Doing so may produce a solution that violates one or more constraints, making it infeasible. Rounding instead to the nearest feasible integer point still fails to guarantee that this solution will be the optimal solution to the MIP.

Example 4.8. Consider the following optimisation problem:

$$\begin{aligned} \max \quad & Z = 2x_1 + 5x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 3, \\ & x_1 + 3x_2 \leq 6, \\ & x_1, x_2 \geq 0, \\ & x_1, x_2 \in \mathbb{Z}. \end{aligned}$$

Figure 4.5 shows that the LP Relaxation gives an optimum solution at $(x_1, x_2) = (1.5, 1.5)$ with objective value

$$Z = 2(1.5) + 5(1.5) = 10.5$$

Rounding both variables to their nearest integers gives $(x_1, x_2) = (2, 2)$ which is infeasible as

$$(2) + (2) = 4 > 3 \quad \text{and} \quad (2) + 3(2) = 8 > 6$$

.

Rounding both variables to the nearest feasible integers gives $(x_1, x_2) = (2, 1)$ with objective value

$$Z = 2(2) + 5(1) = 9$$

However, the feasible solution $(x_1, x_2) = (0, 2)$ has objective value

$$Z = 2(0) + 5(2) = 10$$

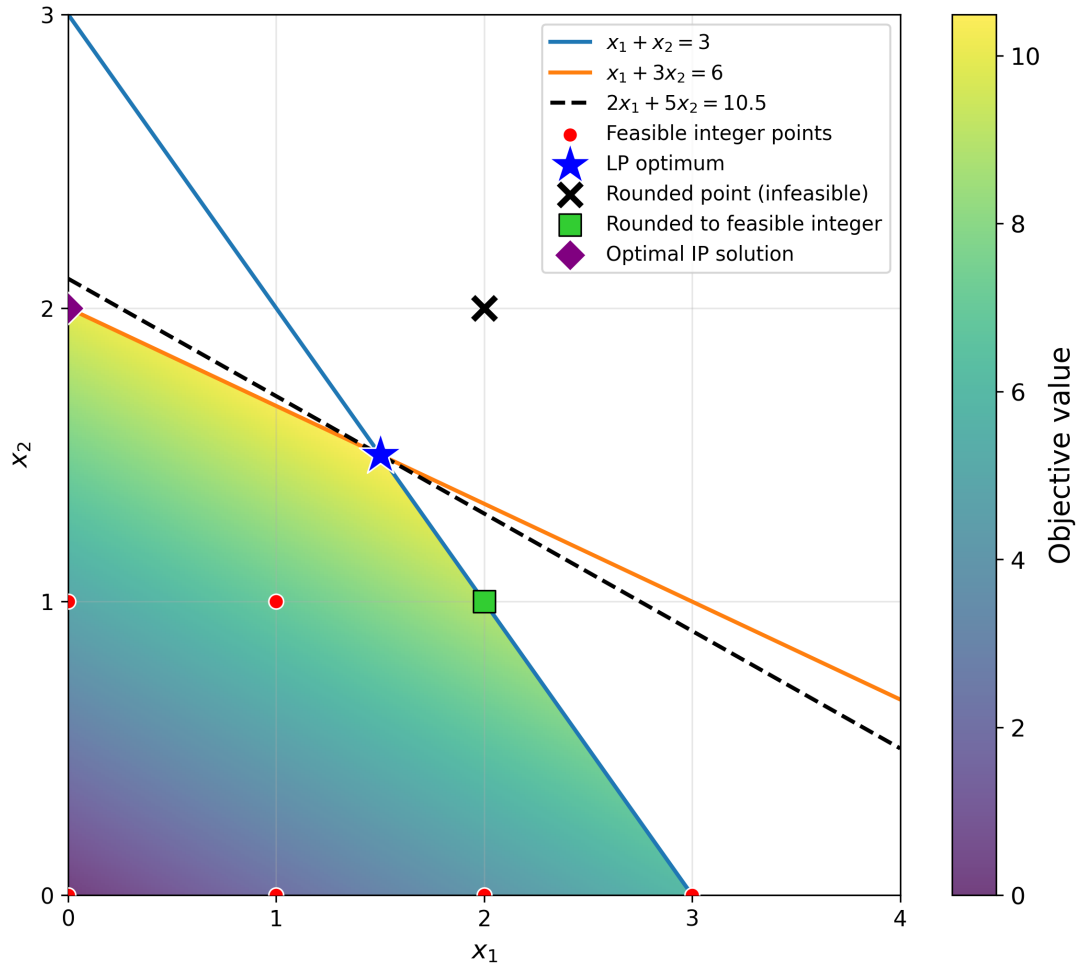


Figure 4.5: LP Relaxation of Example 4.8 demonstrating the problem with round-
ing non-integer solutions to MIP relaxations

Therefore, we need a different approach to find the optimum solution of the MIP problem. This is where we turn to Branching and Bounding.

4.2.2 Method

Branch and bound (B&B) is a method for solving mixed-integer optimisation problems by systematically exploring a set of subproblems. These subproblems are organised in a tree structure, known as the *branch-and-bound tree*.

Each node in the tree corresponds to a subproblem. The root node represents relaxation of the original problem. This is solved and then additional constraints are applied to get child nodes, which represent smaller subproblems. This continues at each stage until an optimal solution is found.

Branching: [11] At a given node k , the LP relaxation of the corresponding subproblem is solved to obtain a solution x_k^{LP} . If this solution is not integer-feasible, a fractional variable x_i^k is selected for branching.

Let $d = \lfloor x_i^k \rfloor$. Two new subproblems are created by adding the constraints

$$x_i \leq d \quad \text{and} \quad x_i \geq d + 1.$$

These define two child nodes in the tree and partition the feasible region while preserving all integer-feasible solutions.

Bounding: [11] At each node, solving the LP relaxation provides an optimal value z_k^{LP} , which acts as a bound on the best solution that can be obtained within that subproblem (see Theorem 4.3).

A node can be 'pruned' in the following cases:

1. **Integer solution:** If x_k^{LP} is integer-feasible, it is a valid solution to the original problem. The incumbent (ie. best current) solution is updated if it improves the objective value.
2. **Infeasibility:** If the LP relaxation is infeasible, then the subproblem contains no feasible solutions.
3. **Bound dominance:** If

$$z_k^{LP} \leq \bar{z},$$

(for a maximisation problem), then the node cannot improve upon the current best solution and is discarded.

The process is repeated recursively, generating a search tree whose nodes are explored until all nodes are either solved or pruned.

Example 4.9. Consider the MILP problem in Example 4.8. From Figure 4.5, the optimal solution of the LP relaxation is $(x_1, x_2) = (1.5, 1.5)$, with objective value

$$Z = 2(1.5) + 5(1.5) = 10.5.$$

Since both variables are fractional, we branch on one of them. Let us branch on $x_1 = 1.5$, creating two subproblems:

$$x_1 \leq 1 \quad \text{and} \quad x_1 \geq 2.$$

The subproblem at Node 1 is shown in Figure 4.7a. Solving this gives the LP optimum $(x_1, x_2) = (1, \frac{5}{3})$, with objective value

$$Z = 2(1) + 5\left(\frac{5}{3}\right) = \frac{31}{3} \approx 10.33.$$

Since x_2 is fractional, we branch again on x_2 , creating the two subproblems

$$x_2 \leq 1 \quad \text{and} \quad x_2 \geq 2.$$

The subproblem at Node 1a is shown in Figure 4.7c. Solving this gives $(x_1, x_2) = (1, 1)$, with objective value

$$Z = 2(1) + 5(1) = 7.$$

This is the first integer-feasible solution, so the node is pruned and this solution becomes the incumbent.

The subproblem at Node 1b is shown in Figure 4.7d. Solving this gives $(x_1, x_2) = (0, 2)$, with objective value

$$Z = 2(0) + 5(2) = 10.$$

This solution is integer-feasible and improves upon the incumbent, so the node is pruned and the incumbent is updated.

The subproblem at Node 2 is shown in Figure 4.7b. Solving this gives $(x_1, x_2) = (2, 1)$, with objective value

$$Z = 2(2) + 5(1) = 9.$$

This solution is integer-feasible but does not improve upon the incumbent, so the node is pruned.

Hence, the optimal integer solution is $(x_1, x_2) = (0, 2)$, with objective value

$$z = 10.$$

Figure 4.6 shows a common way of presenting the Branch and Bound method.

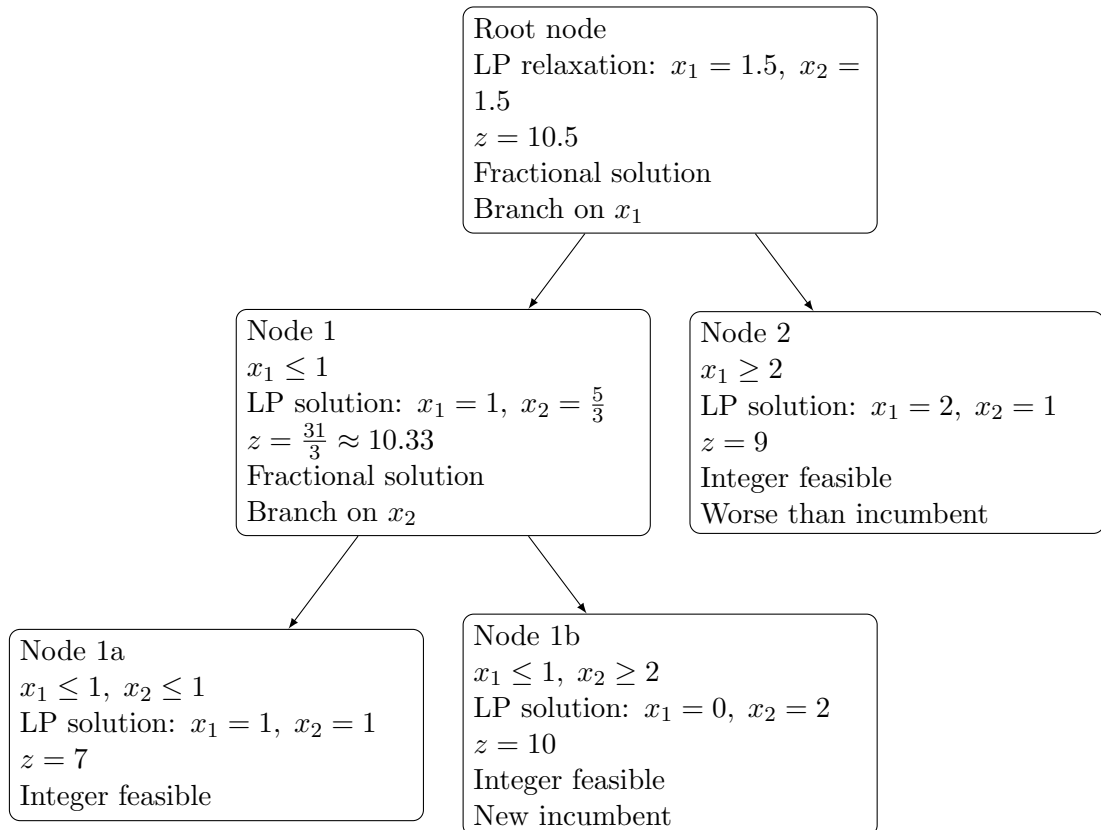


Figure 4.6: Branch-and-bound tree obtained by branching first on x_1 .

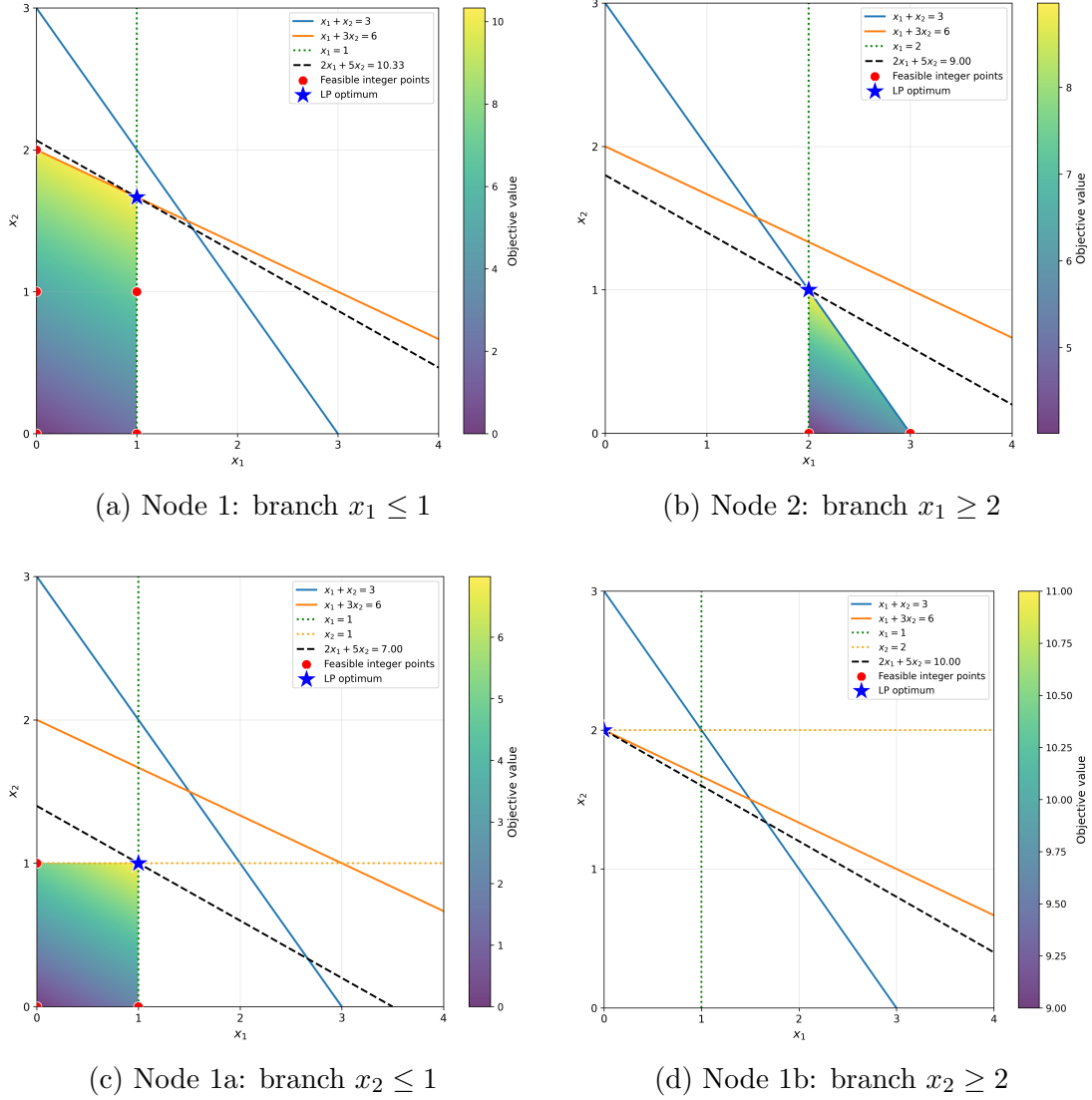


Figure 4.7: Geometric interpretation of the branch-and-bound procedure for Example 4.9.

4.3 MIP Solvers

Branch and bound remains the foundation of many modern mixed-integer optimisation solvers. One prominent example is Gurobi, which will be used in later chapters to solve instances of the HSTT problem. It is therefore useful to outline, at a high level, how such solvers operate. [9]

Modern solvers extend the basic branch-and-bound framework by incorporating additional techniques. These include but are not limited to presolve, cutting planes and heuristics. Presolve reduces the model size, cuts improve bounds and heuristics help identify good feasible solutions early in the search.

Presolve: This happens before the solving begins. It removes redundant constraints and tightens variable bounds. The result of this is a smaller model which in turn can improve computational efficiency

Cutting planes: These are extra constraints which are added to the LP relaxation in order to eliminate fractional solutions without removing any feasible integer solutions. They strengthen the relaxation and thus improve the quality of bounds.

Common examples include rounding-based cuts and Gomory cuts (see [11] for further details)

Heuristics: Whilst cutting planes improve the quality of the LP relaxation by tightening bounds, heuristic methods operate in a complementary manner. Rather than strengthening the relaxation, heuristics aim to find good feasible solutions early in the search process. Though they cannot guarantee optimality, they are often highly effective in practice, especially where time is limited.

Some common examples of heuristic methods in MIP problems include:

- **Rounding heuristics:** Construct integer solutions by rounding fractional LP solutions and checking feasibility.
- **Local search:** Start from a feasible solution and iteratively improve it by exploring neighbouring solutions.
- **Diving heuristics:** Follow a single branch deeply by fixing variables based on promising LP solutions.

By improving the incumbent objective value $\bar{z}$, more nodes satisfy the pruning condition

$$z_k^{LP} \leq \bar{z},$$

and can therefore be discarded. As a result, the size of the search tree is reduced, leading to improved computational performance.

Other Techniques Modern solvers will also exploit parallel computing, where different processors explore different parts of the search tree simultaneously. These processors share information such as best incumbent solution is shared globally, allowing for more efficient pruning of nodes.

4.4 MIP Difficulties

While modern MIP solvers have achieved remarkable success through the integration of branch-and-bound, cutting planes, and heuristic methods, their performance remains fundamentally constrained by the combinatorial nature of integer optimisation. Some of the key challenges of working with MIPs are listed below:

Computational Complexity and Speed Mixed-integer programming problems are, in general, NP-hard [8]. This means that the time required to solve a problem can grow exponentially with the size of the input.

Branch-and-bound methods guarantee optimality, but can only do so once all nodes have either been explored or pruned. Problems can quickly become computationally infeasible and this motivates the use of heuristic methods.

Memory Consumption Large branch-and-bound trees use large amounts of memory to store nodes, constraints, and intermediate solutions. As problems become large, memory can become a constraint.

Formulation Sensitivity The performance of a MIP solver is highly sensitive to how the problem is formulated. Different formulations of the same problem can lead to vastly different computational performance, particularly in terms of relaxation strength and solver efficiency.

Part II

High School Timetabling

Chapter 5

The High School Timetabling Problem

5.1 Problem Definition

To formally define the High School Timetabling (HSTT) problem, we first introduce its fundamental components.

Definition 5.1. [22] An **event** is an activity that must be scheduled. It can be interpreted as the assignment of a set of resources to a timeslot.

Definition 5.2. [22] A **resource** is an entity that participates in events. A **resource group** is a collection of related resources. For example, a teacher such as Mr Ryan is a resource belonging to the Teachers resource group, and a class such as Class 1A is a resource belonging to the Classes resource group. A resource may belong to multiple resource groups simultaneously (e.g., Mr Ryan may also belong to the Maths Teachers group).

Definition 5.3. [22] A **timeslot** is a discrete unit of time in which an event may be scheduled. A **time group** is a collection of related timeslots. For example, a timeslot may belong to both a specific day (e.g., Monday) and a broader grouping such as Week 1.

In practice, an event may represent either a single lesson or an aggregate activity such as a course. For instance, a mathematics course may require multiple teaching periods distributed across a week. This motivates the introduction of sub-events.

Definition 5.4. [22] A **sub-event** is a component of a parent event. The total duration of all sub-events must equal the duration of the parent event.

We can now define the High School Timetabling (HSTT) problem.

Definition 5.5. [26] The High School Timetabling (HSTT) problem is the task of assigning events (or sub-events) to timeslots and resources such that:

- no resource is assigned to more than one event in the same timeslot,

- *all hard constraints are satisfied, and*
- *the total penalty associated with violations of soft constraints is minimised.*

The HSTT problem shares similarities with the Allocation problem discussed in Chapter 3. However, it is fundamentally different. The objective of that allocation problem was to assign a resource (classrooms) to entities (classes) subject to the constraints. Whilst this does capture part of the HSTT structure, (e.g. the assigning of classrooms to events), it does not account for temporal considerations.

Timetabling is a scheduling problem. This means that assignments must also respect the ordering and continuity of time. For example, events (or sub-event) spanning multiple periods must be scheduled in consecutive periods. This introduces temporal constraints that are absent in traditional allocation problems and significantly increases the complexity of the model.

The HSTT problem also exhibits a number of fundamental differences from other educational scheduling problems. In particular, unlike examination timetabling, events in HSTT are recurring and must be distributed across multiple timeslots. High school schedules are also significantly more constrained than University schedules. Safeguarding rules limit the allowance for idle periods for students during the school day. Moreover, schools tend to operate with more limited resource flexibility and a higher density of scheduled activities per student or class.

These characteristics result in a dense and highly interdependent decision space, where both resource allocation and temporal structure must be considered simultaneously. Consequently, any modelling approach must account not only for assignment constraints but also for sequencing and continuity of events over time.

5.2 Problem Modelling and Objectives

The aim of the subsequent chapters is to develop a Mixed Integer Programming (MIP) formulation for real instances of the High School Timetabling (HSTT) problem that is capable of producing high-quality feasible timetables while maintaining computational tractability. In particular, emphasis is placed on model size, solution time, objective value, and the ability to scale across diverse problem instances.

In practical timetabling environments, obtaining a provably optimal solution is often less important than generating a high-quality feasible solution within a reasonable time frame. This is certainly the case with School timetables, where decision-makers are primarily concerned with the usability and quality of the timetable rather than guaranteed optimality. Consequently, this work prioritises formulations that balance solution quality with computational efficiency, rather than those that focus solely on optimality.

The model is implemented in Python using Pyomo for formulation, and solved using the Gurobi Optimizer (version 13.0.0). To ensure fair and reproducible computational comparisons, all experiments are performed using a single solver thread unless explicitly stated otherwise. This approach reduces variability due

to parallel execution and aligns with common benchmarking practices in other optimisation studies [14].

A time limit of 10 minutes is imposed on each solve in order to reflect practical usage scenarios. While some solution approaches involve multiple stages, the majority of computational effort is typically concentrated in a single stage, with subsequent stages requiring comparatively negligible time.

The modelling approach follows an iterative refinement process. An initial formulation is developed to address simpler instances of the HSTT problem and establish baseline performance. The formulation is then progressively strengthened to address common challenges encountered by exact optimisation methods (see Chapter 4.4), with modifications evaluated against the baseline. These improvements are subsequently incorporated into a final formulation capable of addressing more complex and general HSTT instances.

To evaluate the effectiveness of the proposed approach, a benchmark dataset is required that captures the structural diversity of real-world HSTT problems. We chose to use the XHSTT dataset.

5.3 The XHSTT Dataset

The XHSTT dataset is a standardised data format designed to represent instances of the High School Timetabling (HSTT) problem. It was introduced to facilitate benchmarking and comparative analysis across a diverse range of real-world timetabling problems [20]. The format is capable of representing both problem instances and their corresponding solutions, and has been used to encode timetabling data from multiple countries.

Each instance in XHSTT is provided as an XML file containing all information required to define a timetabling problem. This includes the set of events, timeslots, and resources, as well as the groups to which these entities belong. In addition, the dataset specifies a set of constraints, including the resources or events to which they apply, their type, and whether they are treated as hard or soft constraints. Penalties associated with soft constraint violations are also defined. As such, each instance fully specifies a HSTT problem that can be directly used for optimisation. For ease of manipulation and integration with the modelling framework, the data was parsed and converted into structured tabular form. A detailed specification of the XHSTT data format is provided in [12].

A key feature of the XHSTT dataset is the diversity of its instances. Different schools structure their timetabling problems in distinct ways, with variations in resource types, event structures, and constraint sets (see Tables 6.2 and 7.2). This diversity makes XHSTT particularly suitable for evaluating the robustness and scalability of optimisation approaches.

The instances considered in this work can be broadly categorised into two classes: *Event-Time* and *Event-Resource-Time* problems. In *Event-Time* problems, the primary task is to assign events to timeslots while avoiding resource conflicts and satisfying all constraints. In contrast, *Event-Resource-Time* problems additionally require the allocation of resources (such as rooms) alongside scheduling decisions. This introduces an additional layer of complexity, as both

temporal and resource assignment decisions must be made simultaneously.

The instances selected for this study are detailed in Tables 6.1 and 7.1. These instances are chosen to reflect the possible complexity of HSTT problems and the variety in their formulations. Small or structurally simple instances are excluded, as they do not provide a meaningful evaluation of model performance. Instead, focus is placed on instances that exhibit a rich combination of constraints, multiple resource types, and non-trivial scheduling requirements. This allows for a more comprehensive assessment of the proposed formulation in realistic and challenging settings.

In the original ITC 2011 timetabling competition, from which the XHSTT dataset is derived [22], both hard and soft constraints were associated with penalty values, with priority given to minimising violations of hard constraints. In contrast, this work treats hard constraints as strict feasibility requirements and considers only soft constraint penalties within the objective function. This modelling choice reflects the practical reality that violations of hard constraints are not acceptable in real-world timetabling settings. As such, emphasis is placed on obtaining feasible solutions of high quality within a MIP framework.

5.3.1 Notation and Problem Structure

For consistency with existing work, we adopt notation similar to that used in [14]. In this section, we introduce the components of the XHSTT instance that are required for the MIP formulation.

We denote the set of events by $\mathcal{E}$ and the set of event groups by $\mathcal{EG}$. The set of timeslots is denoted by $\mathcal{T}$ and the set of time groups by $\mathcal{TG}$. Furthermore, we denote the set of resources by $\mathcal{R}$ and the set of constraints by $\mathcal{C}$.

Each event $e \in \mathcal{E}$ has a duration $D_e \in \mathbb{N}$ and is associated with a set of event-resources. An event-resource represents a requirement that a resource must be assigned to the event. Some event-resources may be preassigned, while others require a suitable resource to be selected during optimisation. Each event-resource may also have an associated role, denoted role_{er} , which is used to link resources to specific constraints.

Events may be divided into sub-events. We let $\mathcal{SE}$ denote the set of all sub-events, and let $se \in e$ indicate that sub-event se belongs to event e . Each sub-event has a duration $D_{se} \leq D_e$, and the total duration of all sub-events associated with an event must equal the duration of the parent event:

$$\sum_{se \in e} D_{se} = D_e.$$

Timeslots are assumed to be ordered chronologically. We let $\rho(t)$ denote the index of timeslot $t \in \mathcal{T}$. A time group $tg \in \mathcal{TG}$ represents a subset of timeslots, and we write $t \in tg$ to indicate that timeslot t belongs to time group tg .

Each constraint $c \in \mathcal{C}$ is of a specific type and applies to a subset of events, event groups, or resources. We use the notation $e \in c$, $eg \in c$, and $r \in c$ to denote that constraint c applies to event $e \in \mathcal{E}$, event group $eg \in \mathcal{EG}$, or resource $r \in \mathcal{R}$, respectively. More generally, we use $p \in c$ to denote an entity to which constraint

c applies.

5.3.2 Constraints and Objective Function

The XHSTT dataset defines 16 categories of constraints. A detailed specification of these constraints is provided in [13]. The instances considered in this work utilise 11 of these constraint types. These are listed in Table 5.1, along with brief descriptions of their effects. A detailed discussion of their implementation within a MIP framework is provided in subsequent chapters.

Constraint	Description
AssignResourceConstraint (ARC)	Ensure events receive required resources.
AssignTimeConstraint (ATC)	Ensure events are assigned a time.
AvoidClashesConstraint (ACC)	Prevent resources attending simultaneous events.
AvoidUnavailableTimesConstraint (AUTC)	Prevent resources scheduled during unavailable times.
ClusterBusyTimesConstraint (CBC)	Restrict the number of distinct time groups in which a resource has scheduled events.
LimitBusyTimesConstraint (LBC)	Restrict the number of scheduled timeslots for a resource within specified time groups.
LimitIdleTimesConstraint (LIC)	Limit idle gaps between a resource's events.
PreferResourcesConstraint (PRC)	Prefer certain resources for event assignments.
PreferTimesConstraint (PTC)	Prefer certain times for scheduling events.
SplitEventsConstraint (SEC)	Restrict how events may be split into sub-events.
SpreadEventsConstraint (SPC)	Spread events of an event-group across timeslots.

Table 5.1: Summary of constraint types in the XHSTT specification.

In this work, hard constraints are modelled explicitly as feasibility requirements, and therefore do not contribute to the objective function. Each soft constraint $c \in \mathcal{C}$ is applied to a set of entities (events, event groups, or resources), denoted $p \in c$. Violations of a constraint are quantified by a non-negative deviation value $\delta_{c,p} \in \mathbb{N}$.

Two types of penalty functions are considered in the instances used in this report:

The **linear penalty function** is given by

$$P_{\text{linear}}(\delta_{c,p}) = \delta_{c,p},$$

while the **quadratic penalty function** is given by

$$P_{\text{quadratic}}(\delta_{c,p}) = \delta_{c,p}^2,$$

where $\delta_{c,p} \geq 0$ denotes the deviation from the constraint.

Each constraint is associated with a weight $\omega_c \geq 0$, reflecting its relative importance. The objective function is defined as the weighted sum of penalties across all soft constraints:

$$f(x) = \sum_{c \in \mathcal{C}} \sum_{p \in c} \omega_c \cdot P_c(\delta_{c,p}),$$

where $P_c(\cdot)$ denotes the penalty function associated with constraint c .

As a result, depending on the choice of penalties, the objective function will either be that of an MILP or a convex MIQP (see Section 4.1.2).

Chapter 6

Event-Time Scheduling Problems

6.1 Problem Description

Event-Time scheduling problems require each event to be assigned to a timeslot such that no resource conflicts occur, all hard constraints are satisfied, and the total penalty associated with soft constraint violations is minimised.

Table 6.1 summarises the key characteristics of the instances considered in this chapter, including the number of timeslots $t \in \mathcal{T}$, resources $r \in \mathcal{R}$, events $e \in \mathcal{E}$, and total event duration $\sum_{e \in \mathcal{E}} D_e$. Table 6.2 indicates which constraint types are present in each instance, and whether they are treated as hard (H), soft (S), both (H/S), or are not present (–).

Dataset	Times	Teachers	Rooms	Students	Classes	# Events	Total Duration
Finland - ArtificialSchool(V1)	20	22	-	-	13	169	200
Finland - College	40	46	34	-	31	387	854
Finland - HighSchool	35	18	13	-	10	172	297
Finland - SecondarySchool	35	25	25	-	14	280	306
Finland - SecondarySchool2	40	22	21	-	36	469	566

Table 6.1: Summary statistics for Event-Time Instances.

Instance	ARC	ATC	ACC	AUTC	CBC	LBC	LIC	LEC	PRC	PTC	SEC	SPC
Finland - ArtificialSchool(V1)	–	H	H	H	–	S	S	–	–	H	H	H
Finland - College	–	H	H	H	–	S	S	–	–	H	H	H
Finland - HighSchool	–	H	H	–	–	S	S	–	–	H	H	H
Finland - SecondarySchool	–	H	H	H/S	–	S	S	–	–	H	H	H
Finland - SecondarySchool2	–	H	H	–	–	S	S	–	–	H	H	H

Table 6.2: Constraint types present in each timetabling instance. Here, H denotes a hard constraint, S denotes a soft constraint, H/S denotes that both hard and soft versions occur, and – denotes that the constraint is not present.

Redundant constraints are omitted from Table 6.2. For example, although the Finland – College instance includes an AssignResourceConstraint in the original specification, all required resources are preassigned. As a result, this constraint does not influence the event-time scheduling problem and is excluded from the analysis.

Note. The instance Finland – ArtificialSchool (V1) is an adapted version of the original XHSTT instance in which room assignments are removed, resulting in a purely event-time scheduling problem.

6.2 Baseline Model

We define our baseline model, to be used on simple HSTT problems. This model assumes that all events have a duration of 1, $D_e = 1$. We define the decision variables, the basic feasible constraints that any generic problem requires and then the instance specific constraints which only some instances need.

6.2.1 Basic Feasible Constraint

These are the constraints that every HSTT problem must enforce in order to solve the most basic HSTT problem.

We first define the decision variable:

$$x_{e,t} = \begin{cases} 1 & \text{if event } e \text{ is scheduled in timeslot } t, \\ 0 & \text{otherwise.} \end{cases}$$

Then we have the following constraints:

Assign Time Constraint Each event (or subevent) must occupy exactly one timeslot:

$$\sum_{t \in \mathcal{T}} x_{e,t} = 1 \quad \forall e \in \mathcal{E}. \quad (6.2.1)$$

Avoid Clashes Constraint A resource can participate in at most one event in any timeslot:

$$\sum_{e \in \mathcal{E}: r \in \mathcal{R}_e} x_{e,t} \leq 1 \quad \forall r \in \mathcal{R}, \forall t \in \mathcal{T}. \quad (6.2.2)$$

6.2.2 Instance Specific Constraints

We next define the non-basic constraints which can be implemented, but are not vital for the most basic HSTT problem. We only define those which are implemented in this chapter.

We define a new decision variable that dictates if an event or resource is allowed to be scheduled at a certain timeslot. It may not be due to an allocated resource being unavailable at certain times for example.

$$a_{e,t} = \begin{cases} 1 & \text{if event } e \text{ is allowed to occupy timeslot } t, \\ 0 & \text{otherwise.} \end{cases}$$

$$a_{r,t} = \begin{cases} 1 & \text{if resource } r \text{ is available in timeslot } t, \\ 0 & \text{otherwise.} \end{cases}$$

Let $\mathcal{R}_e \subseteq \mathcal{R}$ denote the set of resources required by event e . Then:

Avoid Unavailable Times Constraint An event may only occur when its allocated resources are available

$$\textbf{Hard: } x_{e,t} \leq a_{r,t} \quad \forall e \in \mathcal{E}, \forall r \in \mathcal{R}_e, \forall t \in \mathcal{T}. \quad (6.2.3)$$

Soft: Let $\delta_{r,t} \in \{0, 1\}$ indicate whether resource r is scheduled in unavailable timeslot t . Then:

$$\delta_{r,t} \geq \sum_{e \in \mathcal{E}: r \in \mathcal{R}_e} x_{e,t} \quad \forall r \in \mathcal{R}, \forall t \in \mathcal{U}_r. \quad (6.2.4)$$

$$\delta_r = \sum_{t \in \mathcal{U}_r} \delta_{r,t} \quad \forall r \in \mathcal{R}. \quad (6.2.5)$$

Prefer Times Constraint An event may only start in a timeslot that is explicitly allowed:

$$x_{e,t} \leq a_{e,t} \quad \forall e \in \mathcal{E}, \forall t \in \mathcal{T}, \quad (6.2.6)$$

Split Events Constraint The number of sub-events into which an event is split must lie within specified bounds:

$$L_e \leq \sum_{t \in \mathcal{T}} x_{e,t} \leq U_e \quad \forall e \in \mathcal{E}. \quad (6.2.7)$$

Spread Events Constraint For each course, the number of events scheduled on a given day must lie between a specified minimum and maximum:

$$\sum_{(e,t) \in \mathcal{X}_{k,d}} x_{e,t} \geq L_{k,d} \quad \forall k \in \mathcal{K}, \forall d \in \mathcal{D}, \quad (6.2.8)$$

$$\sum_{(e,t) \in \mathcal{X}_{k,d}} x_{e,t} \leq U_{k,d} \quad \forall k \in \mathcal{K}, \forall d \in \mathcal{D}. \quad (6.2.9)$$

where $\mathcal{X}_{k,d}$ is the set of all (event, timeslot) pairs:

$$\mathcal{X}_{k,d} = \{(e, t) \in \mathcal{E} \times \mathcal{T} \mid \kappa(e) = k, \delta(t) = d\},$$

Let $q_{r,t}$ be the number of events involving resource r , scheduled at time t :

$$q_{r,t} = \sum_{e \in \mathcal{E}: r \in \mathcal{R}_e} x_{e,t} \quad \forall r \in \mathcal{R}, \forall t \in \mathcal{T},$$

Then it indicates which timeslots a resource is busy:

Limit Busy Times Constraint (Soft) This constraint limits the number of timeslots a resource is active within a given time group (e.g., a day).

Let

$$B_{r,d} = \sum_{t \in \mathcal{T}_d} q_{r,t} \quad \forall r \in \mathcal{R}_g, \forall d \in \mathcal{D}.$$

Deviations from bounds $[L_g, U_g]$, are defined as

$$u_{r,d} \geq L_g - B_{r,d}, \quad (6.2.10)$$

$$o_{r,d} \geq B_{r,d} - U_g, \quad (6.2.11)$$

with $u_{r,d}, o_{r,d} \geq 0$.

Limit Idle Times Constraint (Soft) This constraint limits the number of idle times (gaps) within a resource's schedule during a given day.

Let

$$B_{r,d} = \sum_{t \in \mathcal{T}_d} q_{r,t} \quad \forall r \in \mathcal{R}, \forall d \in \mathcal{D}.$$

Let $f_{r,d}$ and $\ell_{r,d}$ denote the first and last occupied timeslots for resource r on day d . The number of idle times is given by

$$I_{r,d} = \ell_{r,d} - f_{r,d} + 1 - B_{r,d}.$$

Deviations from bounds $[L_g, U_g]$, are defined as

$$u_{r,d} \geq L_g - I_{r,d}, \quad (6.2.12)$$

$$o_{r,d} \geq I_{r,d} - U_g, \quad (6.2.13)$$

with $u_{r,d}, o_{r,d} \geq 0$.

6.3 Proposing a Two-Stage Model

The baseline formulation exhibits several limitations. Firstly, it only accommodates events of unit duration, and therefore cannot directly model events spanning multiple consecutive timeslots. Secondly, including both hard and soft constraints in a single formulation can create a large model. This increases computational difficulty, as the solver must satisfy all hard constraints while optimising a more complex objective.

The main challenge in finding feasible solutions arises from the combinatorial nature of the problem. Hard constraints restrict the feasible region, while soft constraints introduce additional variables and dependencies that increase problem size and search complexity. As a result, branch-and-bound methods may spend significant time exploring infeasible or low-quality regions before identifying a feasible solution.

To address these challenges, Kristiansen et. al. [14] proposes a two-stage modelling approach. In the first stage, only the feasibility problem is solved, considering hard constraints alone. This produces a feasible timetable without regard to soft constraint penalties. In the second stage, the full optimisation problem is solved, using the feasible solution from the first stage as a warm start.

A warm start provides the solver with an initial feasible solution by preloading variable values. This can significantly improve computational performance, as it allows the solver to begin the search from a known feasible point, leading to stronger bounds and reducing the need to explore infeasible regions of the search space.

Following this approach, we adapt our model to a two-stage framework. In particular, the feasibility stage allows us to construct valid schedules for each instance before introducing additional complexity through soft constraints and extended modelling features, such as handling events with longer durations.

6.4 Events Spanning Multiple Timeslots

There are three natural approaches to extending the baseline model so that it may account for events that span across multiple consecutive timeslots. The first is an *occupancy-based* formulation. In this model, we decide directly which timeslots an event occupies, using

$$w_{e,t} = \begin{cases} 1 & \text{if event } e \text{ occupies timeslot } t, \\ 0 & \text{otherwise.} \end{cases}$$

Additional constraints are then required to ensure that the occupied timeslots form a single continuous block of the correct length. These constraints must also rule out illegal blocks, such as those that cross day boundaries, as a lesson cannot begin on a Monday afternoon and finish on a Tuesday morning.

The second approach is a *start-time* formulation. Rather than deciding which timeslots an event occupies, we decide only where it starts. We now let:

$$x_{e,t} = \begin{cases} 1 & \text{if event } e \text{ starts at timeslot } t, \\ 0 & \text{otherwise.} \end{cases}$$

and let:

$$a_{e,t} = \begin{cases} 1 & \text{if event } e \text{ is allowed to start at timeslot } t, \\ 0 & \text{otherwise,} \end{cases}$$

Then the following constraint enforces feasibility of start times.

$$x_{e,t} \leq a_{e,t} \quad \forall e \in \mathcal{E}, \forall t \in \mathcal{T}.$$

Parameter $a_{e,t}$ encodes whether a start time is legal, taking into account factors such as event duration, day boundaries, and any relevant resource availability. If $a_{e,t} = 0$, then the event cannot start at time t . In this way, the legality of multi-period events is enforced directly at the level of start times, rather than through many separate occupancy constraints.

The final approach is a *hybrid* formulation. This uses both start variables $x_{e,t}$ and occupancy variables $w_{e,t}$, together with linking constraints.

This can be viewed as a start-time model with explicit occupancy variables added. The advantage is that some constraints become easier to express. For

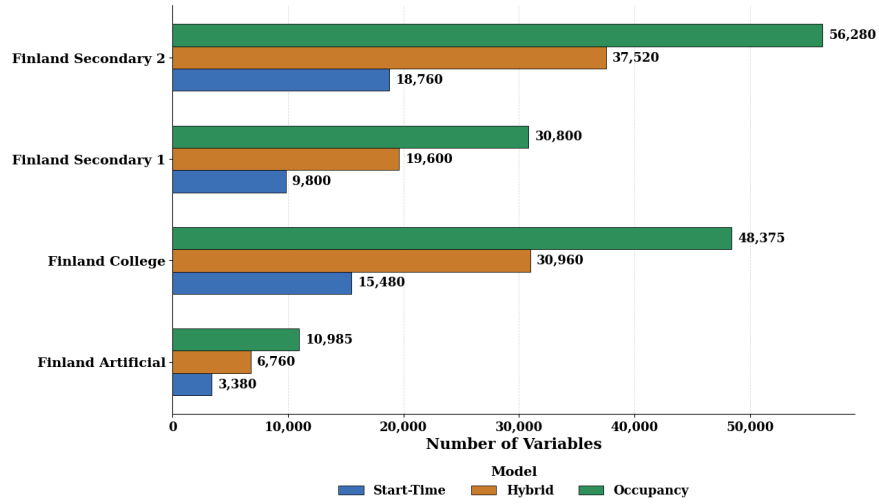
example, teacher or class clash constraints can be written directly in terms of $w_{e,t}$, rather than summing over all start times that would cause an event to occupy timeslot t . The disadvantage is that the formulation introduces more variables and linking constraints, which may increase solve time.

Hypothesis 1. *The start-time formulation results in the smallest model and enables the fastest identification of feasible solutions.*

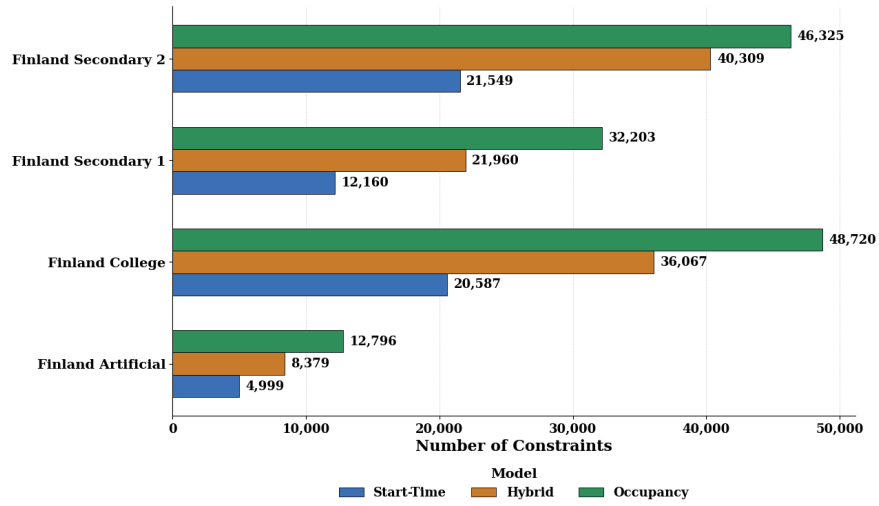
The start-time formulation is expected to be the strongest for fixed-duration events because start-time legality is enforced at the choice of start time. By contrast, the occupancy model must recover legality indirectly through additional constraints, which tends to produce a larger formulation. The hybrid model retains some of this additional structure, and so is expected to lie between the two.

To test this, each formulation was run ten times and the results were averaged. The results, shown in Figure 6.1, demonstrate that the occupancy-based model is consistently the largest, with more than three times as many variables and more than twice the number of constraints as the start-time model. The hybrid model lies between the two. This is in line with expectations, since the occupancy model requires extra structure to enforce continuity and exclude illegal blocks, whereas the start-time model builds these conditions directly into the feasible start set.

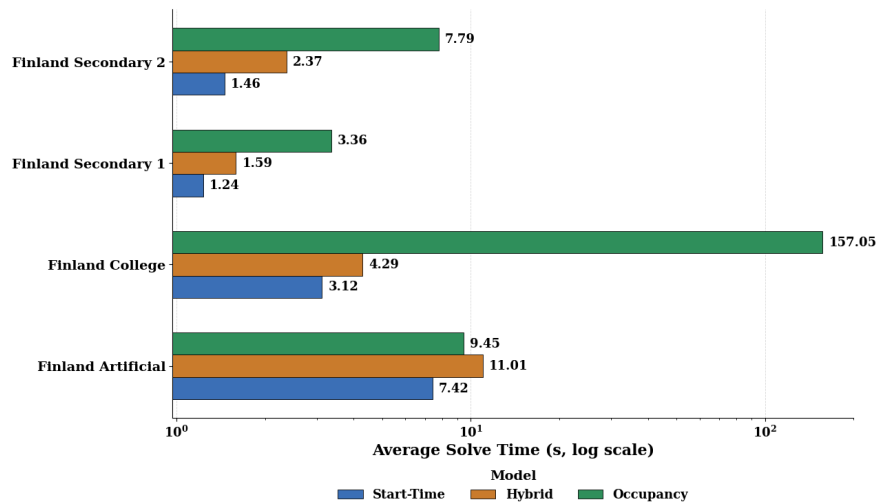
The same pattern is reflected in the solve times. In every instance, the start-time model solved the feasibility problem fastest. The hybrid model was usually close behind, while the occupancy model was significantly slower. For example, in the Finland College instance, the occupancy model required an average of 157 seconds, compared with 3.12 seconds for the start-time model and 4.29 seconds for the hybrid model. This strongly supports our belief that the smaller and tighter start-time formulation provides a substantial computational advantage for fixed-duration event-time problems.



(a) Number of variables for different instances



(b) Number of constraints for different instances



(c) Average solve time for different instances (log scale)

Figure 6.1: Analysis of different occupancy formulations for events spanning across multiple timeslots

6.5 Feasible Times

A key limitation of the baseline formulation is that infeasible event-time assignments are handled explicitly through constraints. This includes restrictions arising from event duration (e.g., multi-period events that cannot extend beyond the end of a day) and resource availability constraints. As a result, many decision variables correspond to assignments that are already known to be infeasible.

In the winning paper of the 2019 International Timetabling Competition, significant emphasis was placed on preprocessing techniques to reduce model size before optimisation. In particular, the use of conflict structures and feasibility filtering has been shown to improve computational performance by eliminating infeasible combinations prior to solving [10].

Motivated by this, we introduce a preprocessing step that restricts each event to a set of feasible start times. For each event $e \in \mathcal{E}$, we define a set $\mathcal{F}_e \subseteq \mathcal{T}$ of feasible timeslots. A timeslot $t \in \mathcal{T}$ is excluded from $\mathcal{F}_e$ if:

- scheduling event e at time t would violate its duration (e.g., extending beyond a day boundary), or
- a preassigned resource of e is unavailable at time t .

The formulation is then modified by restricting decision variables to $t \in \mathcal{F}_e$, rather than all $t \in \mathcal{T}$. This replaces certain hard constraints with implicit feasibility enforced during preprocessing.

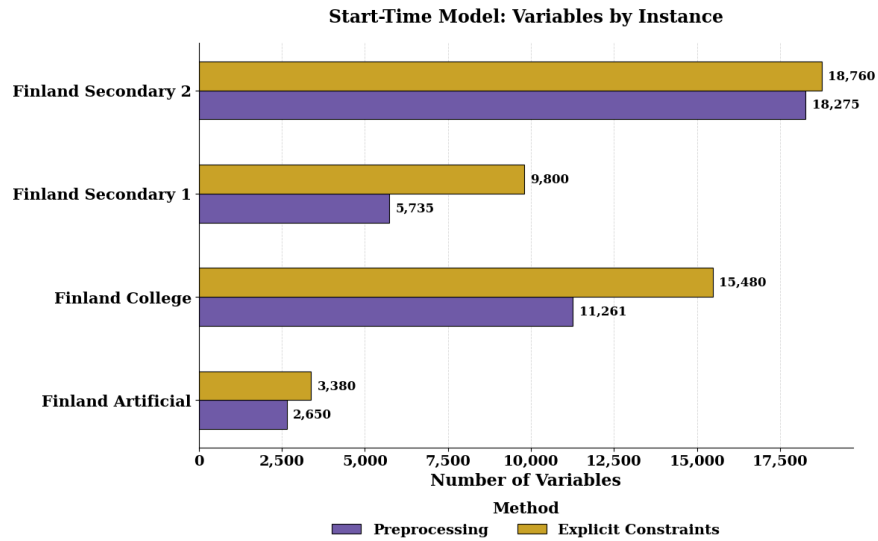
Hypothesis 2. *Feasible-time preprocessing reduces model size and improves solve performance.*

We again ran each model 10 times and took averages of the results. By restricting each event to its feasible start times, the number of start-time decision variables reduces from $|\mathcal{E}||\mathcal{T}|$ to $\sum_{e \in \mathcal{E}} |\mathcal{F}_e|$, with a corresponding reduction in constraint count. This leads to a more compact formulation.

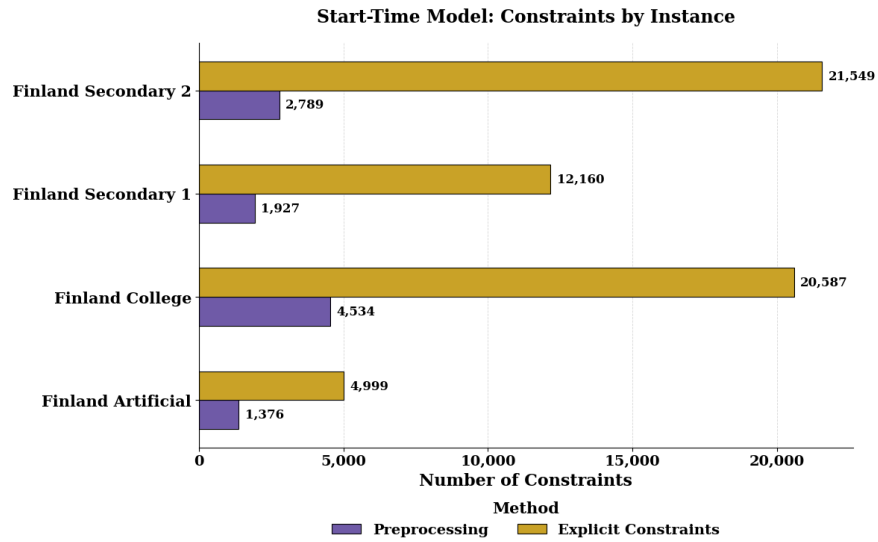
Empirically, Figure 6.2 shows that preprocessing leads to a substantial reduction in model size, most noticeably in the number of constraints which has shrunk in some instances by nearly an order of magnitude. This occurs because infeasible assignments are removed before model construction, eliminating the need for constraints that would otherwise forbid them.

However, a smaller model does not always lead to faster solve times. In some cases, solve time increases slightly; for example, in the Finland Artificial instance, the average solve time increases from 7.42 seconds to 11.52 seconds. This suggests that model size alone does not fully determine performance. While preprocessing reduces the number of variables and constraints, it may also alter the structure of the problem in ways that affect the solver’s search behaviour.

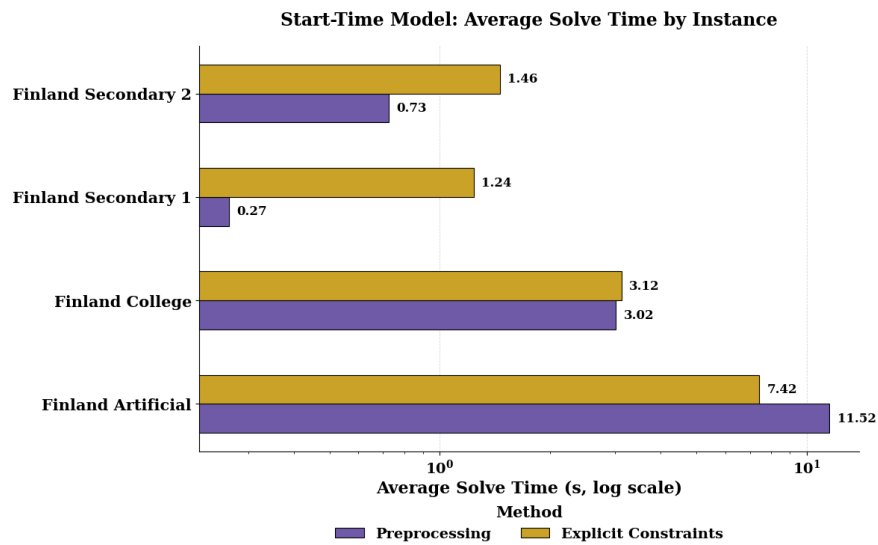
Nevertheless, preprocessing consistently produces more compact and simpler formulations by removing impossible start times in advance. This reduces the number of candidate variables and the density of constraint matrices, which can improve the efficiency of linear relaxations and branch-and-bound search in many cases.



(a) Number of variables for different instances



(b) Number of constraints for different instances



(c) Average solve time for different instances (log scale)

Figure 6.2: Analysis of preprocessing feasible times vs enforcing explicit constraints

6.6 Two-Stage Model Analysis

We evaluate the performance of the two-stage framework introduced in Section 6.3 against a single-stage formulation in which hard and soft constraints are solved simultaneously. Both models use a start-time formulation and use feasible-time preprocessing.

Hypothesis 3. *A two-stage approach, separating feasibility and optimisation, improves feasibility discovery time and solution robustness compared to a single-stage formulation.*

Computational results in Figure 6.3 strongly support this hypothesis. Within the imposed time limit, the combined model was only able to find feasible solutions for two instances, whereas the two-stage approach successfully identified feasible solutions for all instances.

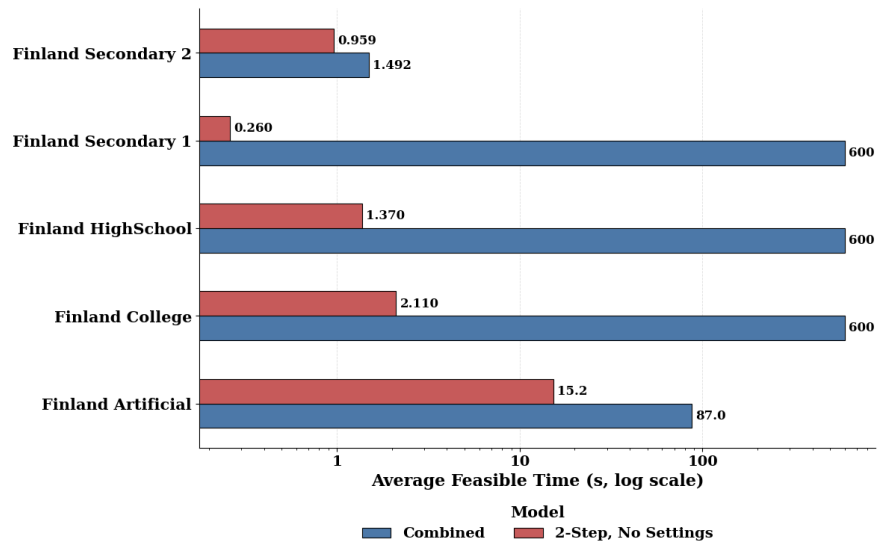
This behaviour reflects structural differences between the two formulations. In the two-stage model, the solver first considers a feasibility problem involving only hard constraints, allowing it to focus solely on identifying a valid solution within a restricted feasible region.

In contrast, the single-stage model introduces additional variables and constraints associated with soft constraints, which increases model size and, more importantly, weakens the linear relaxation and introduces additional coupling between decisions. As a result, the solver must navigate a more complex search space, which can delay the discovery of a feasible solution.

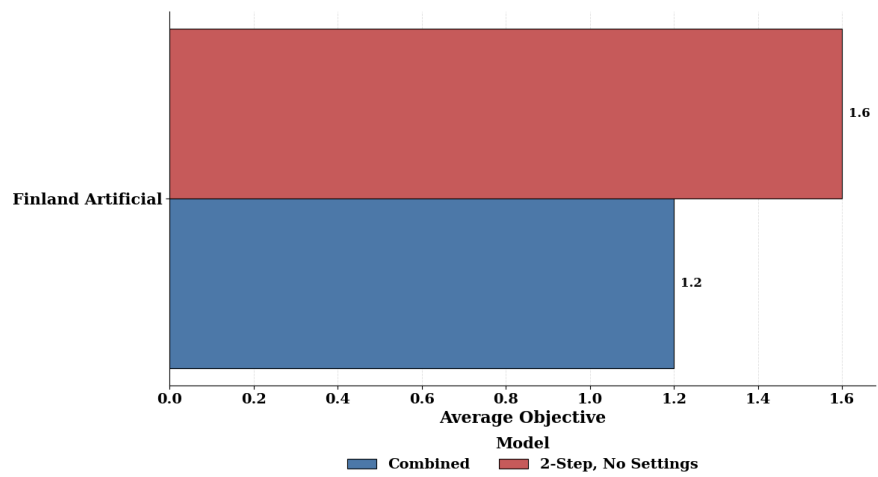
The rapid detection of a feasible solution is highly desirable in timetabling applications. The two-stage approach provides a reliable mechanism for enabling this. However, rapid detection of a feasible solution is not enough alone to guarantee a better optimal solution. Indeed in one of the instances in which the single-stage formulation did find a feasible solution, it far outperformed the two-stage model.

From a practical perspective, rapid feasibility detection is highly desirable in timetabling applications. The two-stage approach provides a reliable mechanism for determining feasibility early, whereas a single-stage model may require significant time before identifying a valid solution.

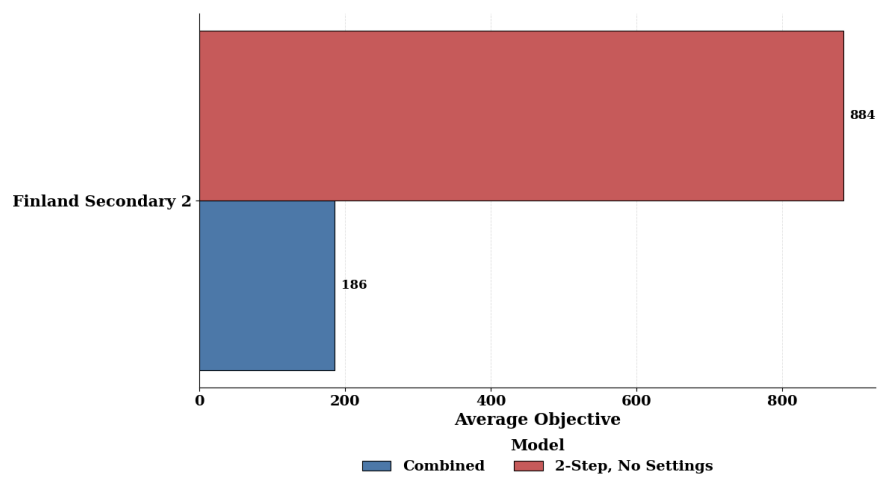
However, the two-stage approach does not always produce better objective values. In the instances where the combined model successfully found feasible solutions, it achieved lower objective values than the two-stage approach.



(a) Average solve time for different instances (log scale)



(b) Average objective values for Finland Artificial (V1) instance



(c) Average objective values for Finland Secondary 2 instance

Figure 6.3: Analysis of 2-stage vs 1-stage model

6.7 Gurobi Settings

The implementation in [14] also considers the impact of solver parameter choices. As in our experiments, a single solver thread is used to ensure reproducibility and reduce variability in computational results.

This work is further motivated by the role of heuristics in modern MIP solvers. As discussed in Section 4.4, Heuristic methods are designed to rapidly identify feasible solutions and improve incumbents early in the search process, even when optimality cannot be guaranteed. In the context of HSTT, where models are large, highly constrained, and solved under strict time limits, this behaviour is particularly desirable.

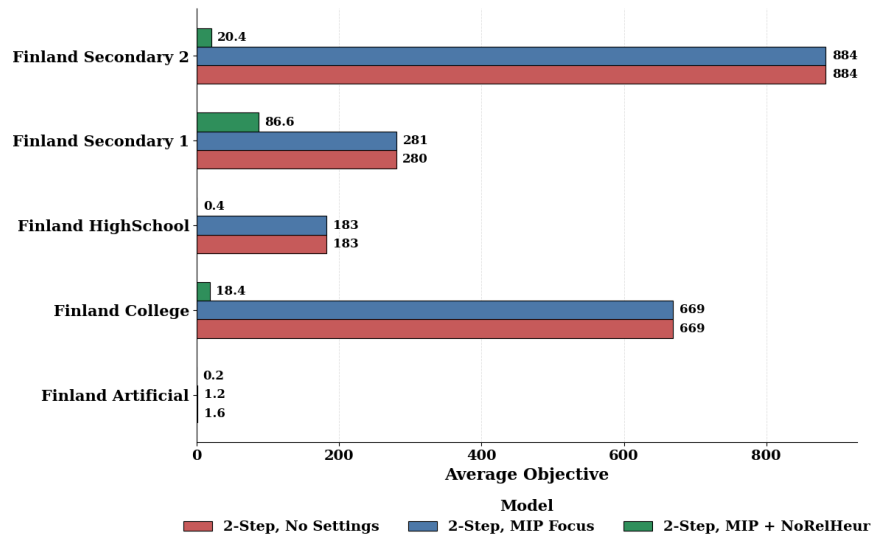
Accordingly, we investigate parameter settings that prioritise heuristic search. The parameter `MIPFocus = 1` directs the solver to emphasise feasible solution discovery by allocating more effort to primal heuristics rather than bound improvement [9]. In addition, we consider a more aggressive configuration using `NoRelHeurTime = 600`, which allows the solver to focus exclusively on heuristic solution methods for the duration of the time limit, without performing root relaxation or branch-and-bound. In this extreme setting, the solver effectively operates as a heuristic method rather than a full MIP solver [9]. This provides an interesting bridge between exact and heuristic approaches, as the solver is restricted to heuristic search while still leveraging MIP-based neighbourhood structures.

We compare three configurations: default settings, `MIPFocus = 1`, and the aggressive heuristic configuration combining `MIPFocus = 1` and `NoRelHeurTime = 600`.

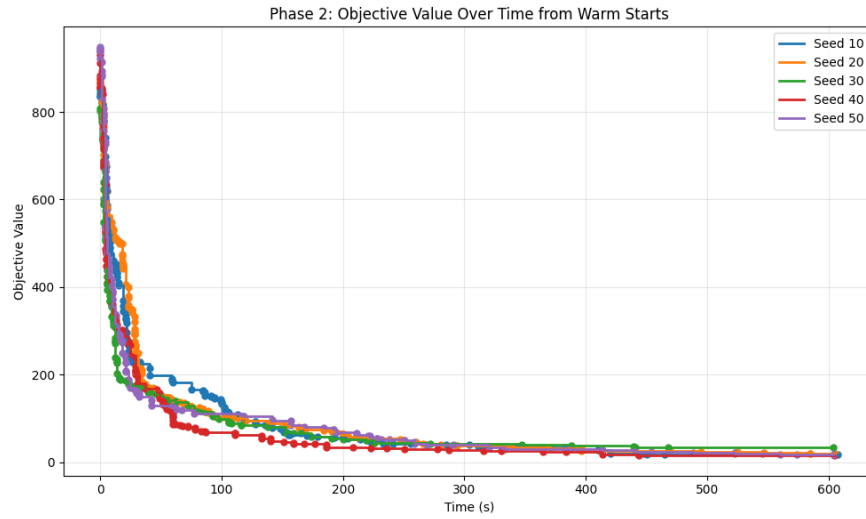
Hypothesis 4. *In a time-limited setting, increasing the emphasis on heuristic search improves the quality of feasible solutions compared to default solver settings.*

Experiments in this example were carried out over 5 runs results were averaged. The results in Figure 6.4 show that setting `MIPFocus = 1` alone yields only modest improvements, and in some cases does not improve solution quality. In contrast, the aggressive heuristic configuration consistently produces better feasible solutions within the time limit.

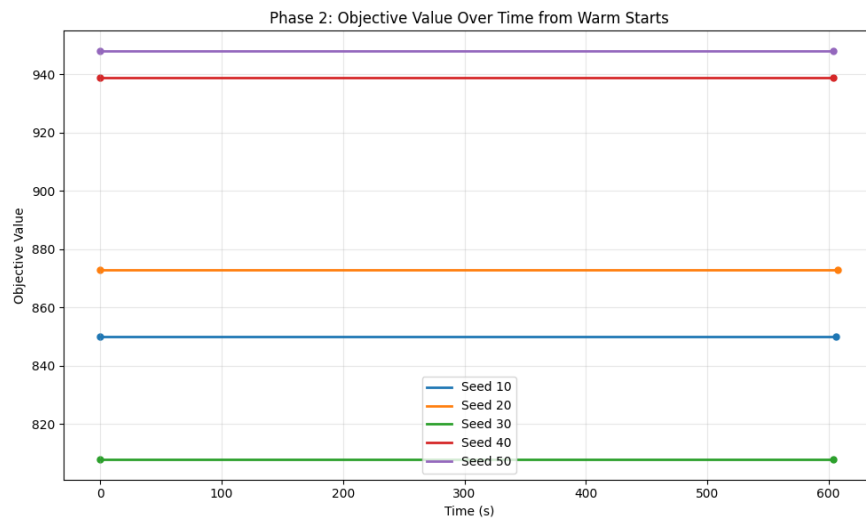
The evolution of the objective solution further illustrates this effect. Under aggressive heuristic settings, most improvements occur early in the search, with only marginal gains later on. This reflects the role of heuristics in quickly identifying high-quality feasible solutions. This behaviour is well aligned with the objectives of high school timetabling, where obtaining a good feasible solution quickly is more important than proving optimality. Consequently, the aggressive heuristic configuration is adopted for the remainder of this work.



(a) Average objective value for different instances



(b) Development of objective value over time for 2-stage model with aggressive heuristic focus



(c) Development of objective value over time for 2-stage model with light heuristics focus

Figure 6.4: Analysis of preprocessing feasible times vs enforcing explicit constraints

6.8 Updated Model

In this chapter, we began with a baseline formulation that explicitly models all constraints and is restricted to unit-duration events. This was extended to handle multi-period events through a start-time formulation, which was shown to produce smaller and more efficient models than both hybrid and occupancy-based approaches.

We then introduced a preprocessing step inspired by techniques used in the ITC 2019 competition [10], restricting events to feasible start times. This reduced the number of variables and constraints, leading to more compact formulations and, in many cases, improved computational performance.

Next, we compared the two-stage approach of [14] with a single-stage formulation. The results demonstrated that the two-stage model is significantly more reliable in time-limited settings, consistently finding feasible solutions where the combined model often failed. This is particularly important in the context of exact methods, where feasibility itself can be difficult to establish (see Section 4.4).

Finally, we explored the impact of solver parameter settings. By combining a two-stage approach with aggressive heuristic configurations in the second stage, we were able to obtain higher-quality feasible solutions within the time limit. This reflects the importance of aligning solver behaviour with the practical objective of timetabling, where rapidly identifying good feasible solutions is often more valuable than proving optimality.

Chapter 7

Event-Resource-Time Scheduling Problems

7.1 Problem Description and MIP Formulation

Event-Resource-Time Scheduling problems require both the scheduling of events to timeslots, and the allocation of these events to resources. Again, the goal is to do this in a way such that resource conflicts are avoided and all hard constraints are satisfied, while minimising violations of soft constraints. In the instances we consider, the resource which events must be assigned to is a classroom.

Table 7.1 summarises the key characteristics of the datasets considered in this chapter and Table 7.2 denotes which constraints are present in each instance, and whether they appear as hard constraints or soft constraints, or both.

Dataset	Times	Teachers	Rooms	Students	Classes	# Events	Total Duration
Denmark - Falkonergaardens	50	90	69	279	-	1077	1077
Finland - ArtificialSchool	20	22	12	-	13	169	200
Finland - ElementarySchool	35	22	21	-	60	291	445
UK - StPaul	25	68	67	-	67	1227	1227
USA - Westside	100	134	108	-	-	628	6354

Table 7.1: Summary statistics for timetabling datasets.

Instance	ARC	ATC	ACC	AUTC	CBC	LBC	LIC	LEC	PRC	PTC	SEC	SPC
Denmark - Falkonergaardens	H	H	H	H	H/S	S	S	-	H	-	-	H/S
Finland - ArtificialSchool	H	H	H	H	-	S	S	-	H	H	H	H
Finland - ElementarySchool	H	H	H	H	-	S	-	-	-	H	H	S
UK - StPaul	H	H	H	-	-	-	S	H	-	-	-	H
USA - Westside	H	H	H	-	-	-	-	-	S	H/S	H	S

Table 7.2: Constraint types present in each timetabling instance. Here, H denotes a hard constraint, S denotes a soft constraint, H/S denotes that both hard and soft versions occur, and - denotes that the constraint is not present.

Extending the model developed in the previous chapter to this setting requires adapting the decision variables and constraints to account for the fact that rooms are not preassigned.

$$x_{e,ts,r} = \begin{cases} 1 & \text{if event } e \text{ starts at timeslot } ts \in \mathcal{F}_e \text{ in room } r \in \mathcal{R}_e, \\ 0 & \text{otherwise.} \end{cases}$$

where $\mathcal{F}_e$ denotes the set of feasible times for event e and $\mathcal{R}_e$ denotes its set of feasible rooms. Note that we extend the preprocessing from the event-time problems to also account for resource feasibility.

Our Basic Feasible constraints then become:

Assign Time and Room Constraint Each event must be assigned exactly one feasible start time and one room:

$$\sum_{\tau \in \mathcal{T}_e^{\text{start}}} \sum_{r \in \mathcal{R}} x_{e,\tau,r} = 1 \quad \forall e \in \mathcal{E}. \quad (7.1.1)$$

Avoid Clash Constraint A resource can be used by at most one event in any timeslot:

$$\sum_{(e,\tau) \in \mathcal{O}_t: k \in \mathcal{K}_e} \sum_{r \in \mathcal{R}} x_{e,\tau,r} \leq 1 \quad \forall k \in \mathcal{K}, \forall t \in \mathcal{T}. \quad (7.1.2)$$

Assign Resource Constraint Each required resource of an event must be assigned exactly once:

$$\sum_{ts \in \mathcal{F}_e} \sum_{r \in \mathcal{R}_{e,k}} x_{e,ts,r} = 1 \quad \forall e \in \mathcal{E}, \forall k \in \mathcal{K}_e. \quad (7.1.3)$$

Further new constraints are included, but omitted here for brevity; for full details, see [14].

7.2 Occupancy Formulations

In the event–time setting, most constraints sum only over feasible start times for each event. In the event–room–time setting, however, these same constraints must typically sum over both feasible start times and feasible rooms. The main source of increased model size is therefore not the constraints themselves, but the expansion of the decision space.

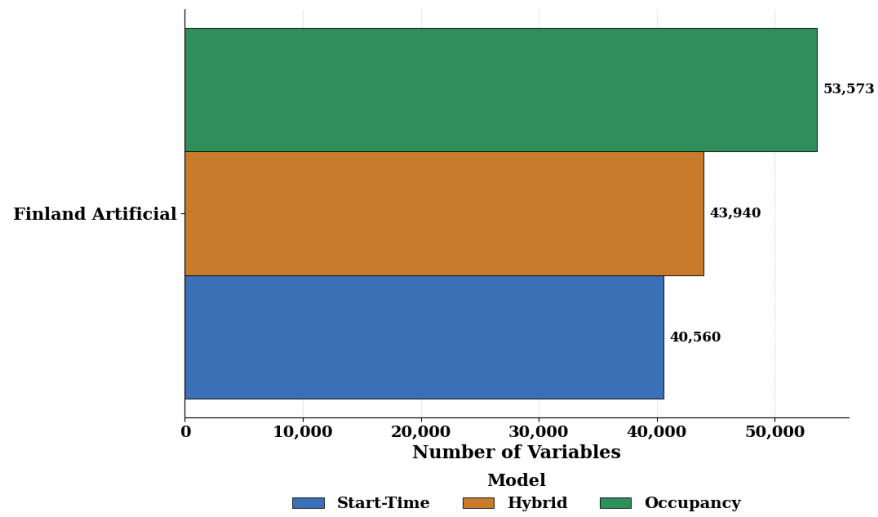
Previously, the formulation considered binary variables indexed by event–time pairs. With joint time–room assignment, the decision variable is instead indexed by event–time–room triples. Hence, whereas the earlier model contained on the order of $|\mathcal{E}| \times |\mathcal{T}|$ binary decision variables, the extended model contains on the order of $|\mathcal{E}| \times |\mathcal{T}| \times |\mathcal{R}|$ binary variables, before preprocessing reductions are applied. This increase in the number of variables is the primary driver of the growth in problem size and has a significant impact on MIP solution times.

In addition, many constraints must now sum over both times and rooms, which increases both the number of constraints and their size. The choice of formulation therefore becomes even more important. Figure 7.1 shows that, once

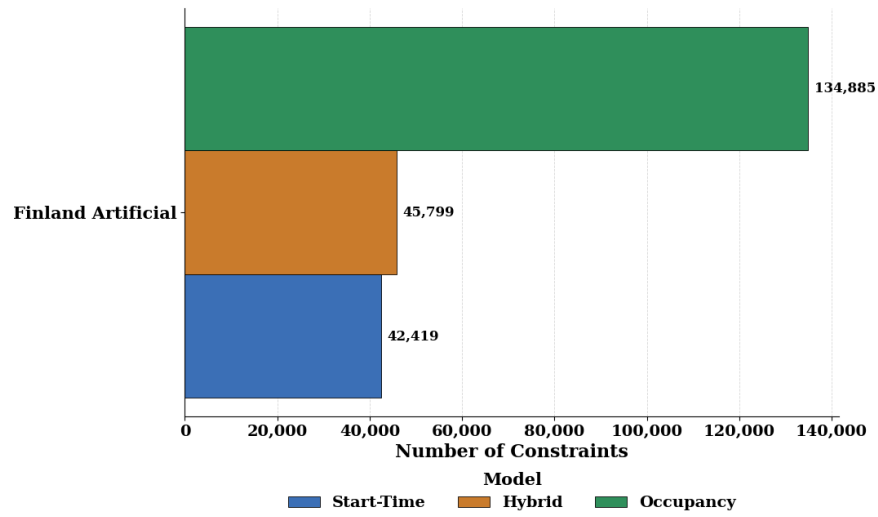
room allocation is introduced, the number of constraints increases substantially across the start-time, hybrid, and occupancy-based formulations. In our experiments, the start-time and hybrid models remain computationally manageable because their formulations are relatively tight, whereas the occupancy formulation becomes much harder to solve, with runtimes increasing sharply.

The results in Figure 7.1 are averaged over five runs. All experiments were carried out using four threads in this case, in order to exploit parallelism during the branch-and-bound search so that solutions could be found in reasonable time.

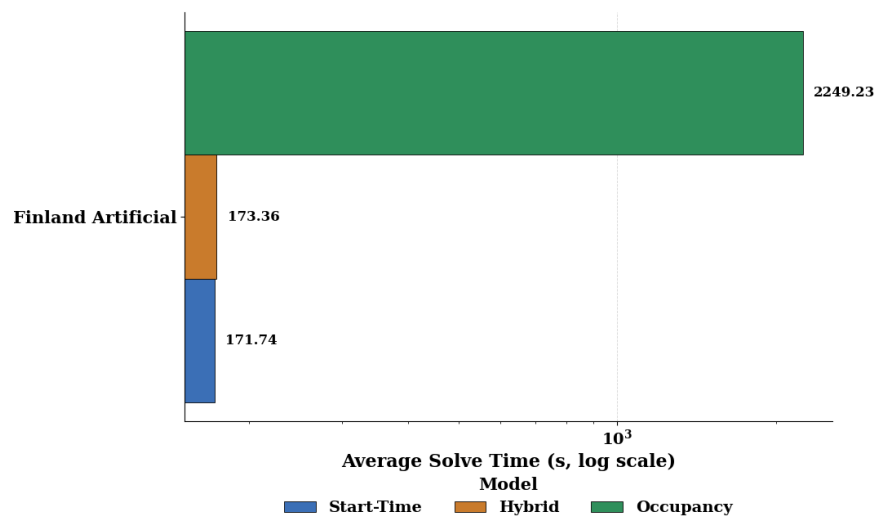
Overall, these results illustrate the computational impact of adding resource allocation to the timetabling problem. Incorporating rooms greatly enlarges the formulation, and this makes the choice of model formulation even more important than in the event-time case.



(a) Number of variables for Finland Artificial instance



(b) Number of constraints for Finland Artificial instance



(c) Average solve time for different (log scale)

Figure 7.1: Analysis of different occupancy formulations for events–resource–time problems

7.3 3-Stage Decomposition

The winning ITC 2019 paper [10] proposed an effective decomposition approach for this problem. Rather than scheduling events to timeslots and assigning rooms simultaneously, they separated the problem into two stages. In the first stage, events are assigned to timeslots. In the second stage, the events, now fixed in time, are allocated to rooms. This yields two substantially smaller models. Whereas the integrated formulation contains, among other variables, on the order of $|\mathcal{E}| \times |\mathcal{TS}| \times |\mathcal{R}|$ binary variables to represent the joint assignment of events to times and rooms, the decomposed approach instead solves two models with variable sets of size $|\mathcal{E}| \times |\mathcal{TS}|$ and $|\mathcal{E}| \times |\mathcal{R}|$, respectively.

This idea is combined with the model developed in the previous chapter to produce a 3-stage decomposition. The first stage solves a feasibility version of the event–time scheduling problem using only hard constraints. The second stage then optimises the event–time schedule, using the stage-one feasible solution as a warm start and incorporating the relevant soft constraints. Finally, the third stage assigns rooms to the timed events.

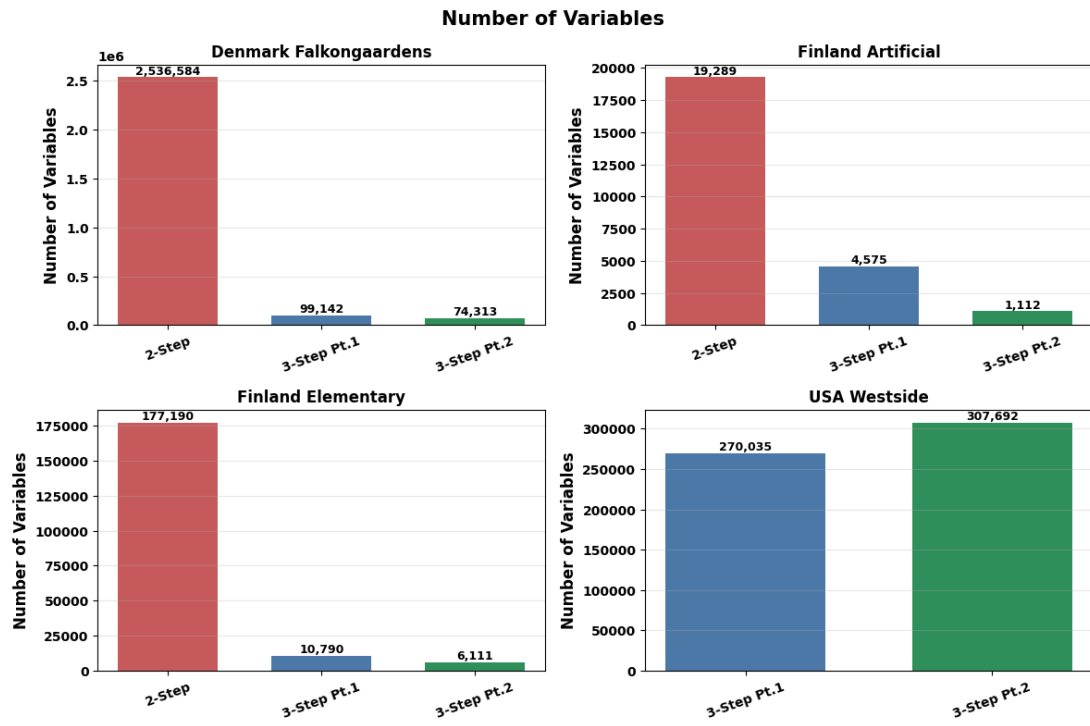
Although the room-assignment stage could itself be further decomposed into a feasibility phase followed by an optimisation phase, this was not adopted here. In most of the instances considered, relatively few soft resource constraints are present, so a single room-assignment model is sufficient.

Hypothesis 5. *Under a fixed computational time limit, the decomposed Event–Resource–Time formulation produces better incumbent solutions than the corresponding integrated formulation in which time and resource assignments are made simultaneously.*

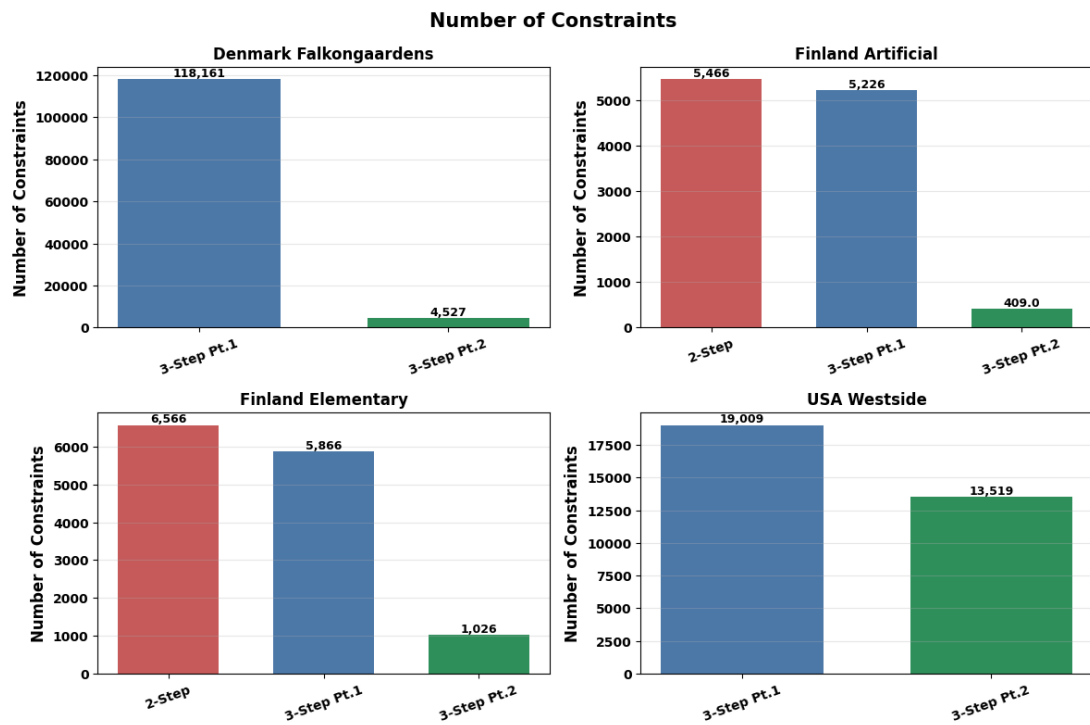
We conducted five runs of each model and aggregated the results. The superiority of the three-stage formulation is evident from Figures 7.2 and 7.3. The three-stage model consistently found a feasible solution in all instances, whereas the two-stage model only succeeded in two cases. In the Denmark Falkongaardens instance, the model ran out of memory, while in the USA Westside instance the $x_{e,t,r}$ variable was too large to construct.

Furthermore, in the USA Westside instance, the objective value for the three-stage model after the event–time stage was 4, but increased by 477 after the event–room allocation stage. This is due to the presence of soft room constraints, which are only accounted for in the later stage.

This highlights a key limitation of the three-stage decomposition: while it improves the ability to find feasible solutions, it may sacrifice overall solution quality by delaying the consideration of certain constraints.

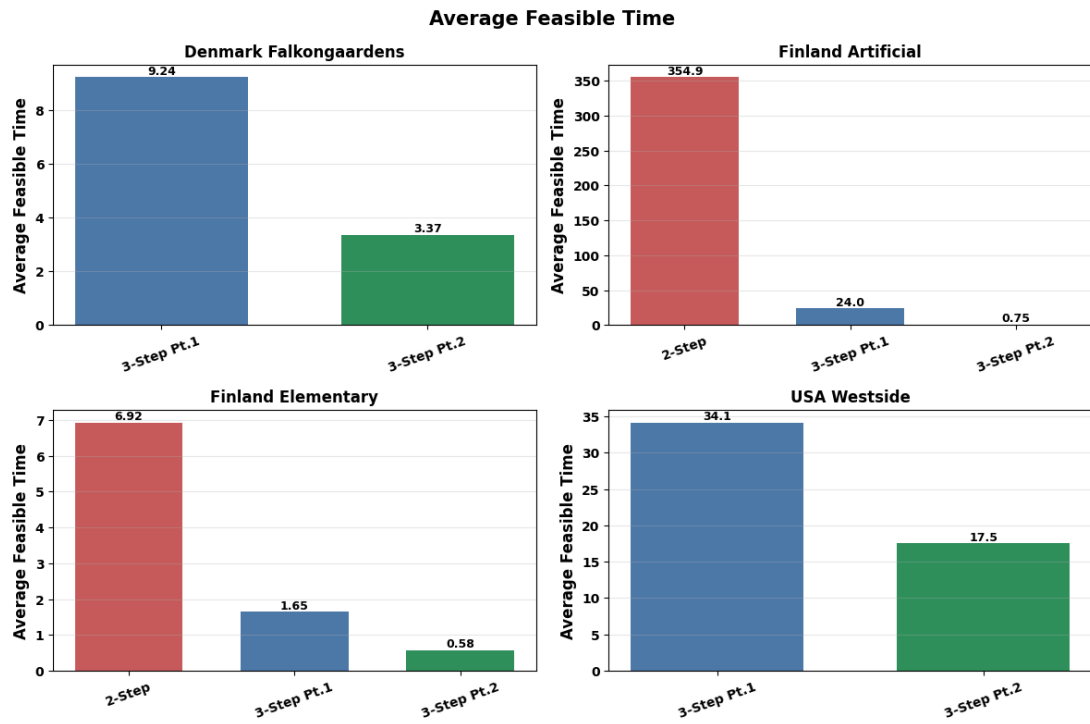


(a) Variables

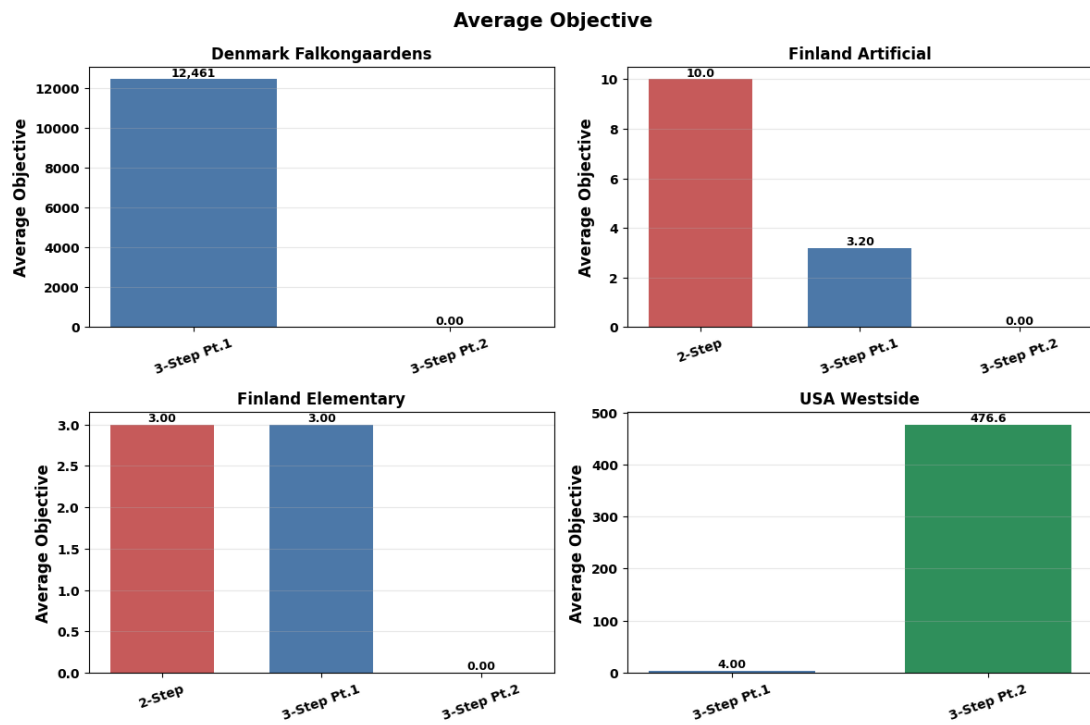


(b) Constraints

Figure 7.2: 2-stage vs 3 stage model size comparison



(a) Time to feasibility



(b) Objective value

Figure 7.3: 2-stage vs 3-stage performance comparison

7.3.1 The Importance of Room Allocation Constraints

Most constraints in timetabling primarily affect event–time decisions rather than resource allocation. This observation underpins the effectiveness of the 3-stage decomposition, as the majority of constraints can be handled before rooms are assigned.

However, this separation introduces a potential issue: the event–time model may produce schedules that are infeasible once room assignment is considered. For example, too many events may be scheduled simultaneously relative to the number of available rooms.

To address this, additional constraints must be included in the event–time stages to limit the number of events assigned to each timeslot based on room availability. These constraints ensure that the final room assignment stage remains feasible and computationally simple.

7.4 Final Model

Building on the developments of the previous chapter, the final model integrates the most effective modelling and solution strategies identified throughout this report.

We begin with the start-time formulation, which was shown to produce more compact and computationally efficient models than both hybrid and occupancy-based approaches. This formulation is further strengthened through preprocessing techniques inspired by the winning paper of the 2019 International Timetabling Competition [10], restricting events to feasible start times and thereby reducing the size of the decision space.

To address the difficulty of finding feasible solutions, the model adopts a three-stage decomposition approach. In the first stage, events are assigned to feasible timeslots while considering only the most critical hard constraints. This stage focuses on establishing feasibility in a reduced problem. In the second stage, soft constraints are added to the event-time problem and the objective is optimised. Finally, in the third stage, resource assignments are introduced, ensuring that all resource-related constraints are also satisfied.

This decomposition separates feasibility from optimisation, significantly improving the reliability of the model in time-limited settings. As demonstrated in the results, the three-stage approach consistently finds feasible solutions in instances where integrated or two-stage formulations fail. However, this comes with a trade-off: delaying the inclusion of certain constraints, particularly soft resource constraints, can lead to reduced solution quality.

To further enhance performance, solver parameters are tuned to prioritise the rapid identification of high-quality feasible solutions. In particular, heuristic strategies are emphasised in later stages, allowing the solver to effectively refine incumbent solutions within the available time.

Overall, the final model represents a balance between formulation strength and computational tractability. By combining preprocessing, decomposition, and solver tuning, it provides a practical and robust approach to solving high-school

timetabling problems using mixed integer programming.

Bibliography

- [1] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004. Available online.
- [2] Michael W. Carter and Gilbert Laporte. Examination timetabling: Algorithmic strategies and applications. *Journal of the Operational Research Society*, 47(3):373–383, 1996.
- [3] Sara Ceschia, Luca Di Gaspero, and Andrea Schaerf. Educational timetabling: Problems, benchmarks, and state-of-the-art results. *European Journal of Operational Research*, 303(1):1–18, 2022.
- [4] Michele Conforti, Gerard Cornuéjols, and Giacomo Zambelli. *Integer Programming*. Springer, 2014.
- [5] Alberto Del Pia, Santanu S. Dey, and Marco Molinaro. Mixed-integer quadratic programming is in np. *Mathematical Programming*, 162(1-2):225–240, 2017.
- [6] Guy Desaulniers, Jacques Desrosiers, Irina Ioachim, Marius M. Solomon, François Soumis, and Daniel Villeneuve. A unified framework for deterministic time-constrained vehicle routing and crew scheduling problems. In *Column Generation*, pages 57–93. Springer, Boston, MA, 1998.
- [7] George H. G. Fonseca, Haroldo G. Santos, Eduardo G. Carrano, and Thomas J. R. Stidsen. Integer programming techniques for educational timetabling. *Discrete Optimization*, 22:39–52, 2016.
- [8] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, San Francisco, 1979. Problem [MP1] Integer Programming.
- [9] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2026.
- [10] Dennis S. Holm, Rasmus Mikkelsen, Matias Sørensen, and Thomas J. R. Stidsen. A graph-based mip formulation of the international timetabling competition 2019. *Journal of Scheduling*, 25:405–428, 2022.
- [11] Jorg Kalcsics. Integer programming: Lecture slides. Lecture slides, University of Edinburgh, 2025/26. Course: Fundamentals of OR.

- [12] Jeffrey H. Kingston. High school timetable data format specification. <http://jeffreykingston.id.au/cgi-bin/hseval.cgi?op=spec>, 2010. Accessed: 2026-03-19.
- [13] Jeffrey H. Kingston. High school timetable data format specification – constraints. <http://jeffreykingston.id.au/cgi-bin/hseval.cgi?op=spec&part=constraints>, 2010. Accessed: 2026-03-19.
- [14] Simon Kristiansen, Matias Sørensen, and Thomas R. Stidsen. Integer programming for the generalized high school timetabling problem. *Journal of Scheduling*, 18(4):377–392, 2015.
- [15] Rhydian Lewis. A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum*, 30(1):167–190, 2008.
- [16] Tobias Müller, Hana Rudová, and Petra Müllerová. The third international timetabling competition (itc 2019): Curriculum-based course timetabling. *Annals of Operations Research*, 2021.
- [17] Keith Murray and Tomáš Müller. Automated system for university timetabling. In *Proceedings of the 6th International Conference on the Practice and Theory of Automated Timetabling (PATAT 2006)*, Brno, Czech Republic, 2006.
- [18] George L. Nemhauser and Laurence A. Wolsey. *Integer and Combinatorial Optimization*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, New York, 1988. Definition of Mixed-Integer Programming, p. 4.
- [19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2nd edition, 2006.
- [20] Gerhard Post, Samad Ahmadi, Sophia Daskalaki, Jeffrey H. Kingston, Jari Kyngäs, Cimmo Nurmi, David Ranson, and Henri Ruizenaar. An xml format for benchmarks in high school timetabling. 2010. Available online.
- [21] Gerhard Post, Samad Ahmadi, Sophia Daskalaki, Jeffrey H. Kingston, Jari Kyngäs, Kimmo Nurmi, David Ranson, and Henri Ruizenaar. An xml format for benchmarks in high school timetabling. In *Proceedings of the 7th International Conference on the Practice and Theory of Automated Timetabling (PATAT 2008)*, Montreal, Canada, 2008.
- [22] Gerhard Post, Luca Di Gaspero, Jeffrey H. Kingston, Barry McCollum, and Andrea Schaerf. The third international timetabling competition. *Annals of Operations Research*, 218(1):69–105, 2013.
- [23] Rong Qu, Edmund K. Burke, Barry McCollum, L. T. G. Merlot, and S. Y. Lee. A survey of search methodologies and automated system development for examination timetabling. *Journal of Scheduling*, 12(1):55–89, 2009.

- [24] E. P. Rau. The emergence of operational research in world war ii. *Endeavour*, 29(4):156–161, 2005.
- [25] Celso C. Ribeiro. Sports scheduling: Problems and applications. *International Transactions in Operational Research*, 19(1-2):201–226, 2012.
- [26] Andrea Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999.